Chapter 1
A Brief Review on Machine Learning



1.1 Machine Learning Definition

The area of Machine Learning (ML) is interested in answering how a computer can "learn" specific tasks such as recognize characters, support the diagnosis of people under severe diseases, classify wine types, separate some material according to its quality (e.g. wood could be separated according to its weakness, so it could be later used to build either pencils or houses). Those and many other applications make evident the usefulness of ML to tackle daily problems and support specialists to take decisions, attracting researchers from different areas of knowledge [1, 17].

In this situation you may ask: but how can a computer learn? Or how can one ensure an algorithm is in fact learning? Before answering such questions we should start by addressing human learning about some subject. Consider a child looking at a chair for the first time ever. How could a child associate the physical object to the concept of a chair? Parents or relatives would first tell him/her that the object being observed is a chair, but (s)he must find a way to extract relevant features to characterize it as a chair. For example: is color an important variable in order to define the concept chair? Well, probably not because one chair could be black, but there are chairs in several other colors. As an alternative, someone would say the shape provides an important feature. Yes, the shape would be indeed a more adequate feature to associate this object to the class or label "chair". By analyzing this simple example, we notice human beings rely on features (also called attributes or even variables) to build up some type of classification function (or simply classifier) f to map an object x into a label y, i.e. $f(x) = y$.

As humans, we are constantly observing different scenarios and attempting to extract relevant features to take conclusions on different tasks. For example, based on the variables of temperature and humidity, we could conclude about the feasibility of playing soccer this evening [8]. We refer to those variables as **input variables** because they will be taken as input to learn some concept (e.g. viability of playing soccer). In this example, the input variables are only associated to weather

© Springer International Publishing AG, part of Springer Nature 2018

R. Fernandes de Mello, M. Antonelli Ponti, *Machine Learning*,
https://doi.org/10.1007/978-3-319-94989-5_1

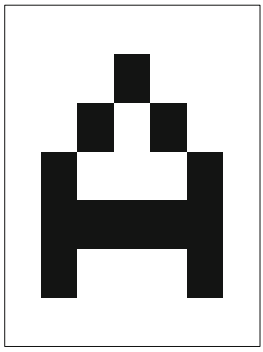


Fig. 1.1 Letter "A" in a bitmap representation

conditions (temperature and humidity), but we could also include health conditions (e.g. past injuries, pain). As in this situation we simply want to predict whether some person will play soccer or not, the **output variable** (a.k.a. class or label) is either positive (yes) or negative (no). Therefore, based on our past experiences and given the current input variables, we must decide whether the output is either yes or no.

There are more complex situations in which the output variable can assume more than two labels. For example, consider a bitmap representing a letter from the alphabet (for instance the letter "A" in Fig. 1.1). Now consider the output variable as an integer value, which assumes 0 for "A", 1 for "B", and so on. This is the sort of situation we have more than two classes. But how could we learn those bitmap representations for the whole alphabet?

To make the computer learn the alphabet representation, we should first build up a **training set** containing n examples. This term "example" is used to refer to the features provided to represent a given letter so it can be learned. For now, let us consider the features as the image of such a letter. We could have letter "A" graphed in different manners, every one constituting one different example (Fig. 1.2 illustrates one example of each letter to be learned). In order to make a simple representation, we will consider letter "A" organized as a binary row vector, as listed in Table 1.1, having 1 as a black and 0 as a white pixel. This vector (a.k.a. **feature vector**) organization is typically used in ML to represent an example. We can see every element of this vector as associated to one input variable, as illustrated in Table 1.2. In this case, each input variable is the pixel value at a given space location (0 or 1). After defining such input variables for our first training example, we add an extra column to our table to represent the output variable, which corresponds to the class (see the last variable in Table 1.2).

Before moving on, we recall a summary of the most important terminology used until now:

1. an **example** (also known as instance or observation) is an observed object of interest (e.g. a letter from the alphabet);
2. a **feature** (also known as attribute or characteristic) is an input variable that is used to characterize the example (e.g. black and white pixels for images);

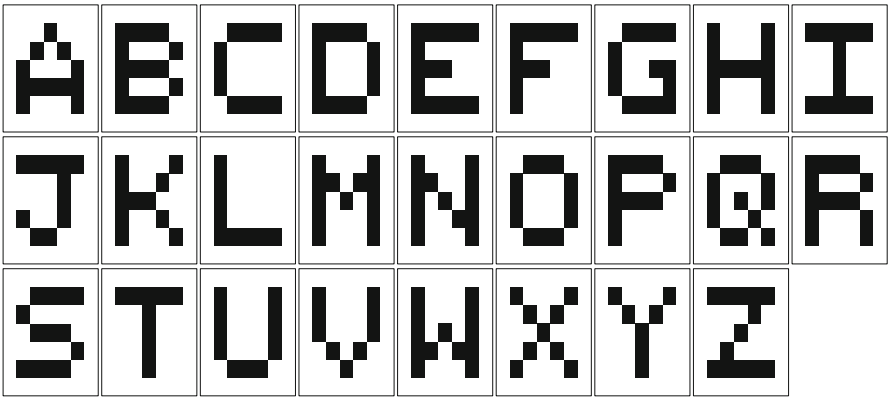


Fig. 1.2 Alphabet in a bitmap representation

Table 1.1 Letter "A" organized as a binary row vector

A:	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	0	1	0	1	1	0	1	1	1	0	1	1	0	0	0	0	0	1	1	0	1	1	1	0	1	1	1	1	1	1	1	1

Table 1.2 Illustrating the variables composing the binary row vector for letter "A", in which P_i refers to the i-th pixel of the bitmap

Letter A:	P_1 1	P_2 1	P_3 1	P_4 1	P_5 1	P_6 1	P_7 1	P_8 1	P_9 1	P_{10} 1	P_{11} 0	P_{12} 1	P_{13} 1	P_{14} 1	P_{15} 1	P_{16} 1	P_{17} 0	
	P_{18} 1	P_{19} 0	P_{20} 1	P_{21} 1	P_{22} 1	P_{23} 0	P_{24} 1	P_{25} 1	P_{26} 1	P_{27} 0	P_{28} 1	P_{29} 1	P_{30} 0	P_{31} 0	P_{32} 0	P_{33} 0	P_{34} 0	
	P_{35} 1	P_{36} 1	P_{37} 0	P_{38} 1	P_{39} 1	P_{40} 1	P_{41} 0	P_{42} 1	P_{43} 1	P_{44} 1	P_{45} 1	P_{46} 1	P_{47} 1	P_{48} 1	P_{49} 1			class 0

3. the **class** of an example is its label typically defined by some specialist of the application domain (e.g. class "A" has label 0);
4. a **classifier** is a function f that maps the set of features x into some label y, therefore it is expected to be capable of classifying examples which were never seen before (typically referred to as unseen examples).

By adding up more rows to Table 1.2, we obtain something similar to Table 1.3, which contains several examples to be learned (one per row). This is the notation considered throughout this book. This data table is commonly referred to as **dataset**, which contains both the input variables (features) and the **supervision**, i.e., the class or label, which was (most probably) defined by a specialist. Datasets like this are provided to a classification algorithm which must infer a classifier $f : X \rightarrow Y$, having $x_i \in X$ as the features of the example at row i and $y_i \in Y$ as the corresponding class to be learned (last column). We expect this algorithm to find the best possible classifier for such a problem. In summary, this is how we typically approach ML tasks involving supervision (i.e., when we have classes) (Fig. 1.3).

Table 1.3 Illustrating a dataset with training examples for the classes 'A' (label 0), 'B' (label 1), 'C' (label 2) and 'D' (label 3)

| Features | | |

Supervised learning has its foundations on the Statistical Learning Theory (SLT), proposed by Vapnik [15, 16], which defines the conditions that learning is ensured. Although the importance of non-supervised learning, it still requires studies to provide stronger theoretical guarantees [3, 14]. This book intends to tackle only supervised learning.

1.3 Supervised Learning

As previously introduced, supervised learning consists in finding the best as possible classifier $f : X \rightarrow Y$ for a given problem. The algorithm responsible for finding this mapping is referred to as **classification algorithm**, which infers a model from every input example $x \in X$ and its respective class $y \in Y$. As we will see in the next chapter, this model is an approximation for the **joint probability distribution** (JPD) of variables X and Y. For now, we are much more interested in discussing about the issues involved when searching for the best classifier f.

As a classifier can be seen as a function $f : X \rightarrow Y$, the easiest way to understand how to obtain the best as possible mapping is probably by considering a regression problem as provided in [13, 17]. In a regression, Y is not a set of discrete labels, but often a range of real values, and, therefore, it will be easier to depict an example for f and understand how well (or how bad) a classifier can be. For instance, consider we want to find a function to map the input variable `atmospheric pressure` to the output class `rain probability` (which is seen as a continuous variable in range $[0, 1]$), as shown in Table 1.4. Thus, our problem is to answer whether it will rain next, according to the current atmospheric pressure measurement. In order to proceed, take a training set of previous observations from which we learn the best function, given the current atmospheric pressure, to produce the probability of raining as output. Listing 1.1 illustrates the program used to generate Table 1.4. Throughout this book, our programs are written to run on the R Statistical Software, provided by the R Foundation for Statistical Computing.[1]

[1]Complete source-codes are available at the repository https://github.com/maponti/ml_statistical_learning. For more information, we suggest the reader to study the R manuals available at https://www.r-project.org.

Table 1.4 Training examples of the regression problem

x	y
63.16	0.28
64.54	0.29
67.62	0.39
57.81	0.14
71.43	0.50
71.17	0.44
62.73	0.27
65.44	0.35
75.71	0.54
64.47	0.32
62.02	0.23
70.92	0.46

Listing 1.1 Read and show regression data

```
# Code to produce the examples of regression problem
# Data columns are located in different files
x <- read.table("atmospheric_pressure.dat")
y <- read.table("rain_probability.dat")

x <- pressure$pressure[7:19]
x <- log(x)
plot(x,y)
```

Figure 1.4 illustrates both dimensions, i.e., the input variable and the output class, for this toy problem. We observe there is some sort of linear relationship between those variables, therefore we could consider a set of linear functions to estimate classifier f. But, by using a linear f, some points of this training set will not coincide with the function, i.e., we will not avoid errors on the training sample, as distances of points from f confirm. So, how about using a polynomial function g as shown in Fig. 1.5. Function g would fit perfectly on this sample. This question motivates the **Bias-Variance Dilemma** from Statistics [17], which is here basically translated to: is it the linear or the polynomial function more appropriate to fit this training sample? To answer that question, we need to quantify appropriateness.

Loss Function This measure of appropriateness requires what is referred to as **loss function**, which is responsible for quantifying error or risk [16, 17]. Two of the most commonly used loss functions are: (1) the 0–1-loss function, and (2) the squared-error loss function. The first basically consists in counting how many times the output class is misclassified by f, as defined in Eq. (1.1). The squared-error loss function quantifies errors in a continuous way such as shown in Eq. (1.2), thus it is adequate when the classifier output is in $\mathbb{R}$, such as in regression problems.

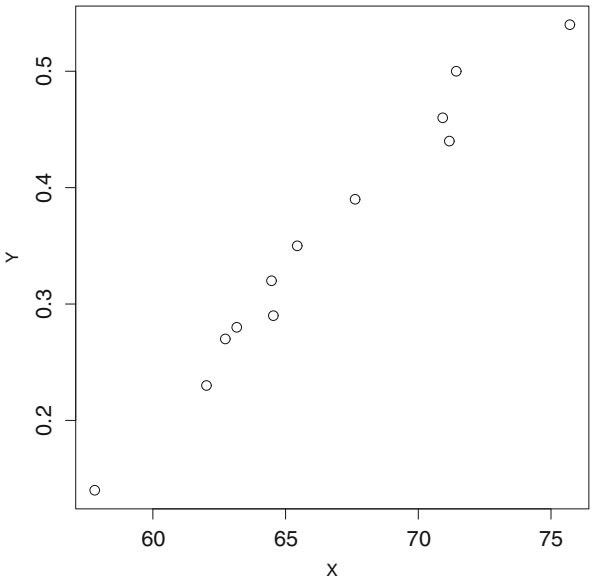


Fig. 1.4 Dataset of pressure x and probability of rain y

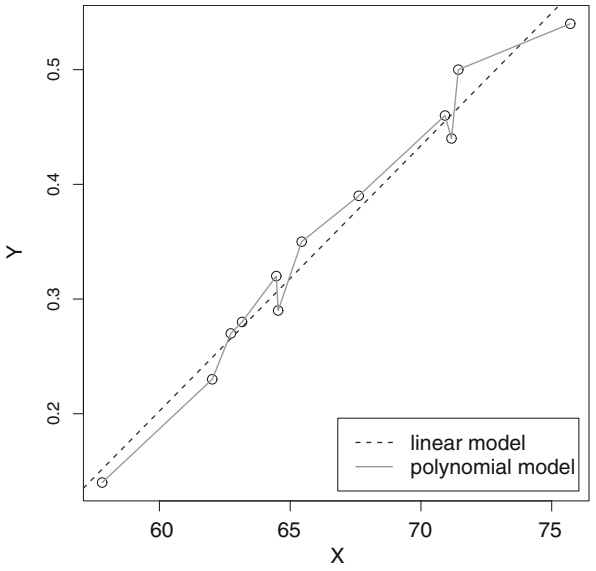


Fig. 1.5 Regression of pressure x and probability of rain y using linear and polynomial functions

$$\ell_{0-1}(X, Y, f(X)) = \begin{cases} 1, \text{ if } f(X) \neq Y \\ 0, \quad \text{otherwise.} \end{cases} \tag{1.1}$$

$$\ell_{\text{squared}}(X, Y, f(X)) = (Y - f(X))^2 \tag{1.2}$$

After selecting the most adequate loss function,[2] we now have a way to quantify appropriateness (in fact, we now leave that word because it may sound subjective and use just "error" or "risk" instead) and then we can get back to the main question: is it the linear or the polynomial function more appropriate to address the problem described earlier (Fig. 1.5)? By only analyzing the loss on the training data, one might say that the polynomial function is better. However, to properly answer this question, we must evaluate how well both classifiers are when predicting unseen examples.

Why unseen examples? Because that allows us to confirm whether classifiers are biased to simply represent training data. For instance, consider two students trying to learn computer programming by themselves. Let us assume the first student memorizes as much as possible book A, while the second has a different strategy: (s)he will avoid memorization and will study using a variety of books as well as coding some applications. Now we consider two tests applied to assess the knowledge acquired by both students:

1. The first test is only based on examples detailed in book A—so the first student would most probably get a perfect score, while the second would get a fair grade;
2. The second test is based on several unseen books (excluding all books both students had seen before), programs and general problems—the second student would probably get a fair score again, while the first would fail miserably. What did it happen? The first student memorized book A!

We probably observed this scenario several times, but which is best? In terms of supervised learning, the capacity of knowledge generalization is best. When generalizing, the student indeed learns a subject and how to apply it on real-world situations rather than just memorize it. How could one measure such achievement?

Risk In the same manner, a classifier f is expected to provide fair results for unseen examples. You can think about it as a test using questions from books the students were not given in advance. To compute how good a classifier is, we must consider the expected value of the loss function as defined (expressed by $:=$) in Eq. (1.3). This measure results in the expected risk $R(.)$ of some classifier f, given any possible example $x \in X$ (including unseen ones) which should be correctly labeled as $y \in Y$. The expected value can be roughly interpreted as the average value of the loss, given every possible input example. Notice this is not feasible for real-world problems which hardly ever provide access to all examples covering the whole space of possibilities.

$$R(f) := E(\ell(X, Y, f(X))). \tag{1.3}$$

[2] Observe one can still define another function instead of using those.

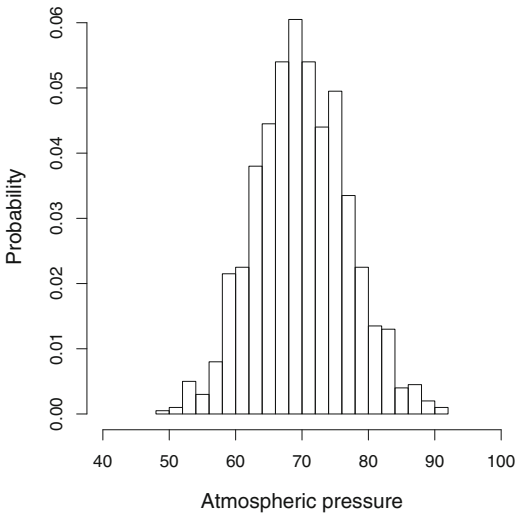


Fig. 1.6 Discretized estimation of the probability distribution function (PDF) of X (atmospheric pressure) considering 1000 measurements

To compute the expected value, we need the **joint probability distribution** (JPD) of variables X and Y so that there is full access to every combination of $x \in X$ versus their corresponding classes $y \in Y$. To introduce the concept of this JPD, consider 1000 measurements of the atmospheric pressure as input variable X, which were collected along years. Figure 1.6 illustrates a discretized estimation of the probability distribution function (PDF) of X considering those 1000 measurements. Let us consider this PDF of X as a good estimation for its real PDF function, as if we had access to the universe of all measurements for such a World region. Then, observe pressure values around 70 kPa are much more probable, thus we will have many more training examples assuming values close to it while inferring classifier f, just because values around 70 kPa are the most common.

Now consider the classes or labels (i.e. rain probability) associated to those 1000 measurements of atmospheric pressure, whose PDF estimative is shown in Fig. 1.7. Observe that both PDFs for X and Y are very close to the Normal distribution (they were in fact synthetically produced using that distribution). The joint probability distribution (JPD) $P(X \times Y)$ is responsible for associating the input examples $x \in X$ to their corresponding output classes $y \in Y$, as illustrated for those 1000 measurements in Fig. 1.8. This is just an example in which the complete JPD is not available, because such distribution would require **every possible** pair (atmospheric pressure, rain probability) and not just 1000 measurements. The actual JPD would require infinite data observations once each variable is in $\mathbb{R}$. In fact, ML supervised algorithms attempt to find the best classifier f fitting $P(X \times Y)$ with the lowest possible error. Now we can infer what most probably happens when a given atmospheric pressure is observed. For instance, consider the current atmospheric pressure is 65 kPa, so we analyze range [64.5, 65.5] (centered at 65 kPa) to check for the most probable value for Y (see Fig. 1.9).

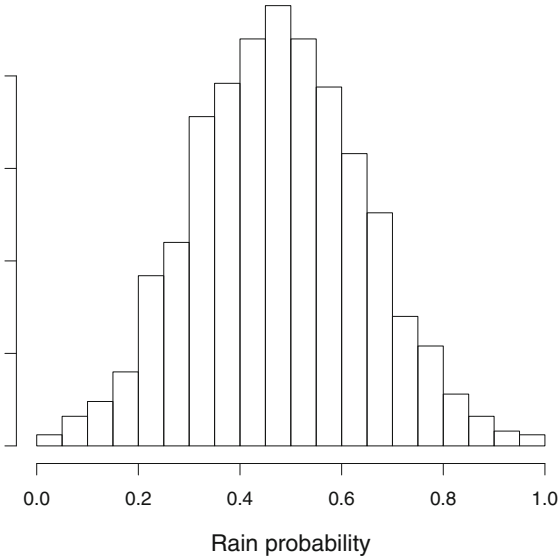


Fig. 1.7 Discretized estimation of the probability distribution function (PDF) of Y (rain probability) considering 1000 measurements

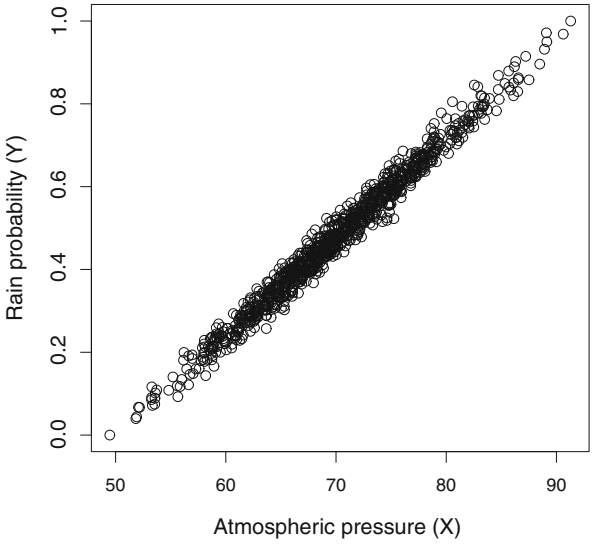


Fig. 1.8 Joint probability distribution (JPD) $P(X \times Y)$ associating the input examples $x \in X$ (atmospheric pressure) to the output classes $y \in Y$ (rain probability) for the 1000 measurements

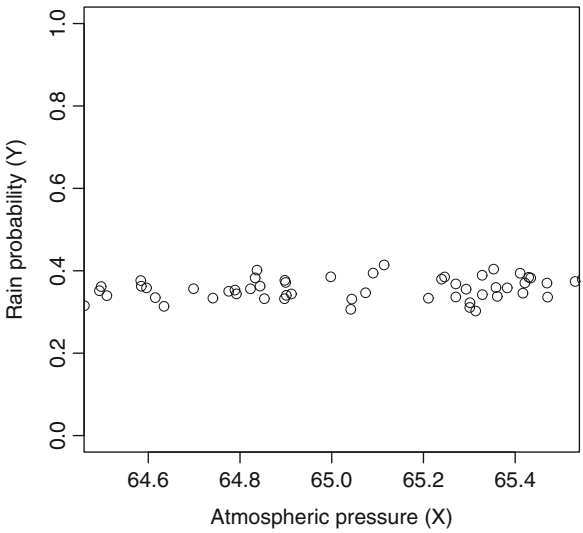


Fig. 1.9 Defining range [64.5, 65.5] for variable X (centered at 65 kPa)

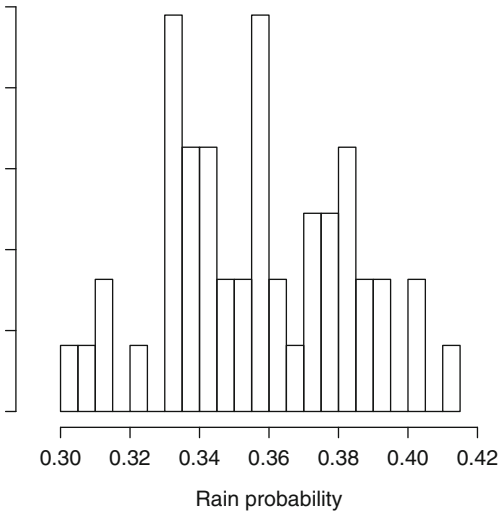


Fig. 1.10 Histogram illustrating the possible values for Y (rain probability), given the range of interest for $X \in [64.5, 65.5]$ (centered at 65 kPa)

This is exactly what the mapping f is expected to produce. Afterwards, we analyze the PDF for Y but only when $64.5 \leq X \leq 65.5$, as illustrated in Fig. 1.10, providing the following probabilities:

$$P(0.27 \leq Y \leq 0.28) = 0.019,$$
$$P(0.28 \leq Y \leq 0.29) = 0.058,$$
$$P(0.29 \leq Y \leq 0.30) = 0.019,$$
$$P(0.30 \leq Y \leq 0.31) = 0.058,$$

$$P(0.31 \leq Y \leq 0.32) = 0.156,$$
$$P(0.32 \leq Y \leq 0.33) = 0.235,$$
$$P(0.33 \leq Y \leq 0.34) = 0.274,$$
$$P(0.34 \leq Y \leq 0.35) = 0.117,$$
$$P(0.35 \leq Y \leq 0.36) = 0.039,$$
$$P(0.36 \leq Y \leq 0.37) = 0.019,$$

from which we conclude the most probable event is in range $[0.33, 0.34]$, as given by $P(0.33 \leq Y \leq 0.34) = 0.274$. So our classifier would provide some value in such range $[0.33, 0.34]$ as output (e.g. its central value 0.335).

This scenario allows us to draw two important conclusions:

1. classifier f estimated with 1000 training examples does not provide a deterministic answer about the correct class. Instead, it provides an answer under some probability (0.274 in this case). So **never expect a classifier to be perfect**, only for very simple toy scenarios;
2. $R(f)$ (see Eq. (1.3)) can never be directly computed for real-world problems, as the real JPD $P(X \times Y)$ is not available. This is only possible when dealing with very simple countable problems.

In practice, classification algorithms attempt to estimate the JPD relying on input examples. The best classifier f is the one that better fits $P(X \times Y)$, and any divergence between f and its JPD implies loss. Therefore, we can formulate risk $R(f)$ as the integral of divergence:

$$\int_{X \times Y} \ell(X, Y, f(X))\, d(X \times Y). \tag{1.4}$$

Of course, there are problems for which the JPD $P(X \times Y)$ has a finite and accessible number of possibilities, consequently $R(f)$ is computable. For example, consider a fair die was thrown and let: (1) the input variable $X = 1$, if an even number occurs (i.e., 2, 4 or 6), and $X = 0$ otherwise; (2) in addition, let a second variable to represent the class or label, having $Y = 1$ if the number is prime (i.e., 2, 3 or 5) and $Y = 0$, otherwise.

In this problem, a single thrown will produce the following joint probabilities:

$$P(X = 0 \times Y = 0) = P(1) = \frac{1}{6}, \text{ probability of observing 1,}$$
$$P(X = 0 \times Y = 1) = P(3, 5) = \frac{2}{6}, \text{ probability of observing 3 or 5,}$$
$$P(X = 1 \times Y = 0) = P(4, 6) = \frac{2}{6}, \text{ probability of observing 4 or 6,}$$

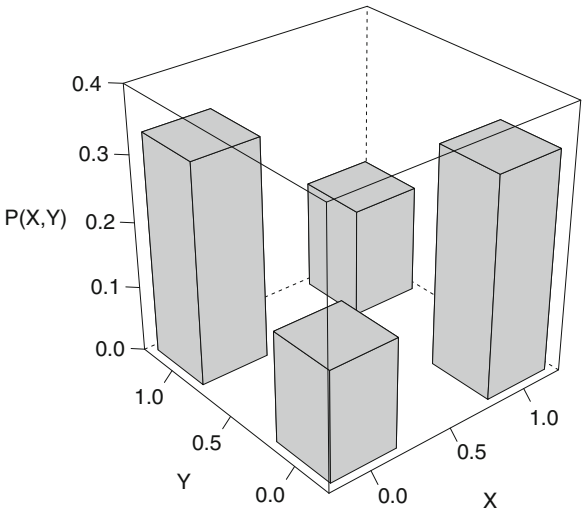


Fig. 1.11 Throwing a die: joint probability distribution for variables X and Y

$$P(X = 1 \times Y = 1) = P(2) = \frac{1}{6}, \text{ probability of observing } 2,$$

which are used to compose the JPD for variables X and Y as illustrated in Fig. 1.11.

In this particular situation, we can compute all necessary probabilities to have the JPD and, because of that, $R(f)$ is computable. This simple situation allows us to assess what is the best possible classifier: any function f that makes the perfect regression over the JPD, as shown in Fig. 1.11. As result, if this classifier receives $X = 0$, it will have two options to select from:

$$P(X = 0 \times Y = 0) = P(1) = \frac{1}{6}, \text{ probability of having value } 1$$

$$P(X = 0 \times Y = 1) = P(3, 5) = \frac{2}{6}, \text{ probability of having either value } 3 \text{ or } 5,$$

and given the most probable is $P(X = 0 \times Y = 1) = \frac{2}{6}$, f should provide $Y = 1$ as output. However, it outputs the incorrect answer for $\frac{1}{6}$ of examples. That allows us to conclude that even such perfect classifier is prone to make mistakes, with an average error of $\frac{1}{3}$ for every input.

For the sake of comparison, consider that any other algorithm converged to another classifier g with the following representation for the same JPD:

$$P_g(X = 0 \times Y = 0) = \frac{0}{6},$$

$$P_g(X = 0 \times Y = 1) = \frac{3}{6},$$

$$P_g(X = 1 \times Y = 0) = \frac{3}{6},$$
$$P_g(X = 1 \times Y = 1) = \frac{0}{6},$$

in this latter situation, the expected risk $R(g)$ is:

$$P(X = 0 \times Y = 0) - P_g(X = 0 \times Y = 0) = \frac{1}{6} - \frac{0}{6} = \frac{1}{6},$$
$$P(X = 0 \times Y = 1) - P_g(X = 0 \times Y = 1) = \frac{2}{6} - \frac{3}{6} = -\frac{1}{6},$$
$$P(X = 1 \times Y = 0) - P_g(X = 1 \times Y = 0) = \frac{2}{6} - \frac{3}{6} = -\frac{1}{6},$$
$$P(X = 1 \times Y = 1) - P_g(X = 1 \times Y = 1) = \frac{1}{6} - \frac{0}{6} = \frac{1}{6},$$

whose sum provides an incorrect value of $R(g) = 0$, due to negative values cancel out positive ones. Thus, we should use the squared-error loss function instead:

$$R(g) = \left(\tfrac{1}{6}\right)^2 + \left(-\tfrac{1}{6}\right)^2 + \left(-\tfrac{1}{6}\right)^2 + \left(\tfrac{1}{6}\right)^2 = \frac{4}{36} = \frac{1}{9},$$

making evident the error of g given unseen examples.

If the JPD is available though, we could compute $R(.)$ for any classifier learned during the training stage. Note that most real-world problems do not fit in this case, that is why **Machine Learning has been widely studied and employed: to estimate good enough classifiers from limited data, without having access to the real JPD**.

Empirical Risk At this point, we are prepared to take a next step. There is no way of computing $R(.)$ assuming $P(X \times Y)$ is unknown at the time of learning. So how could one measure the loss of classifier f and conclude about its usefulness for some problem? We start by estimating the risk based only on the training examples, given they are sampled from the same JPD, as defined in Eq. (1.5). This estimator is referred to as the **empirical risk** of classifier f.

$$R_{\text{emp}}(f) := \frac{1}{n} \sum_{i=1}^{n} \ell(X_i, Y_i, f(X_i)) \tag{1.5}$$

Can one use this risk estimative to select the best classifier? This question is central for the SLT and it is not that simple to answer. To investigate whether the empirical risk $R_{\text{emp}}(.)$ can be used as a good estimator for risk $R(.)$, Vapnik considered the **Law of Large Numbers**, one of the most important theorems from Statistics [5]. However, he faced many challenges to employ this law, requiring a

set of assumptions about input examples. To begin with, the Law of Large Numbers (Eq. (1.6)) states the mean of some random variable Ξ converges to the expected value of the underlying distribution P as the sample size n goes to infinity, provided Ξ_i is drawn in an independent and identical manner from P, i.e.:

$$\frac{1}{n}\sum_{i=1}^{n}\Xi_i \rightarrow E(\Xi), \text{ for } n \rightarrow \infty. \tag{1.6}$$

Observe this theorem is very close to what Vapnik needed. So, by showing that the empirical risk $R_{\text{emp}}(.)$ converges to risk $R(.)$ for any classifier f, then one could select the best candidate among them (see Eq. (1.7)).

$$\frac{1}{n}\sum_{i=1}^{n}\ell(X_i, Y_i, f(X_i)) \rightarrow E(\ell(X_i, Y_i, f(X_i))), \text{ for } n \rightarrow \infty \tag{1.7}$$

Recall that the best classifier is the one providing minimal risk $R(.)$. However, there are several **assumptions to be held**:

1. The joint probability distribution $P(X \times Y)$ is unknown at the time of training;
2. No assumption is made about the joint probability distribution $P(X \times Y)$;
3. Labels may assume nondeterministic values;
4. The joint probability distribution $P(X \times Y)$ is static/fixed;
5. Examples must be sampled in an independent and identical manner;

The first assumption states the joint probability distribution $P(X \times Y)$ is unknown at the time of training, as a consequence the supervised algorithm must estimate it without any previous knowledge but only using samples drawn from this JPD. Observe $P(X \times Y)$ is estimated using an indirect access to itself, once training examples provide a pointwise perspective of this distribution. If the joint probability distribution is known, one could simply estimate its parameters by using some fitting process. By assuming the JPD is unknown, learning would be possible given any data probability distribution.

The second assumption states the joint probability distribution $P(X \times Y)$ may come from any family of distributions. It can either be Normal, Poisson, Binomial or any other [12]. This assumption makes impossible the selection of a given family to later estimate its parameters. As a consequence, the Statistical Learning Theory works in an agnostic way about $P(X \times Y)$.

In the third assumption, labels may have nondeterministic values. The first reason is that data collection may impose noise to labels. For instance, suppose a person is classifying customers from a Web commerce application and misses the correct label in some situations. It should not be impossible to learn even in the presence of such eventual errors. The second reason for this assumption is that classes may overlap, e.g. when throwing a die, as previously seen. For example, the classification of people according to sex, given their height as input variable X [17]. Such a variable is not sufficient to completely separate male and female classes apart,

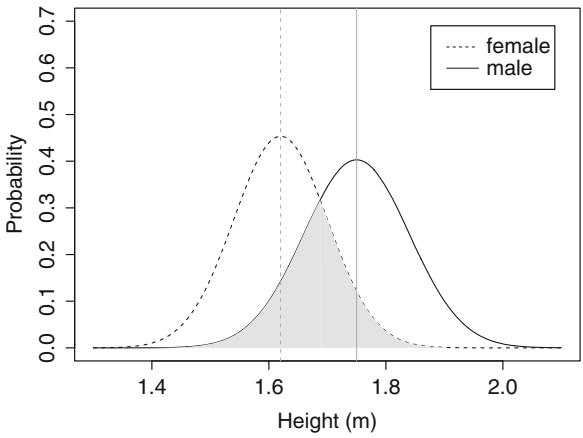


Fig. 1.12 Probability distributions of male and female according to height in meters, with vertical lines indicating the mean of the distributions

especially due to some height intervals as illustrated in Fig. 1.12. It is difficult to find a good classifier for this problem once there is a significant overlap of class distributions, as depicted in the grey area.

This third assumption is very important because data may contain incorrect labels or a mixture of them. To exemplify this mixture, consider Fig. 1.13 which shows a $P(X \times Y)$ for this problem involving heights. A classification would bisect the heights into two regions, but there is still a significant overlap. Compare it with Fig. 1.12, which shows heights versus densities. To improve classification results, we should consider more meaningful variables to compose the input set X.

The fourth assumption states the joint probability distribution $P(X \times Y)$ is static, i.e., it never changes along time. This is necessary because the empirical risk (Eq. (1.5)) is required to approximate $R(.)$ as the sample size n tends to infinity, otherwise variations in the mapping from X to Y would cause conflicts to ensure such approximation. It is obvious that if the JPD starts representing a given association from X to Y and then changes it, all previous training is useless. Time series and data stream applications suffer in such a context, as data behavior may change and therefore impact in the joint probability distribution.

As last assumption, examples must be statistically independent from each other and be sampled in an identical manner. To illustrate, see Fig. 1.14 in which a bag contains three balls enumerated as $1, 2$ and 3. Now consider the probability of drawing either ball $1, 2$ or 3 which is equal to $P(\text{ball} = 1) = P(\text{ball} = 2) = P(\text{ball} = 3) = 1/3$. After drawing ball 1 **without replacement**, the probabilities change to $P(\text{ball} = 1) = 0$, $P(\text{ball} = 2) = 1/2$ and $P(\text{ball} = 3) = 1/2$. This happens because the event of drawing ball 1 influences the probability of future events, what is assumed not to occur in the context of the Statistical Learning Theory. Therefore, the SLT considers the sampling **with replacement** from the JPD. As part of this last assumption, examples must be sampled in an identical manner, meaning each one of the three balls must have the same probability of being chosen, consequently there is no bias to define one of them as preferred.

All those assumptions are necessary to employ the Law of Large Numbers, which is used in the context of the SLT to ensure the empirical risk $R_{\text{emp}}(.)$ is a good

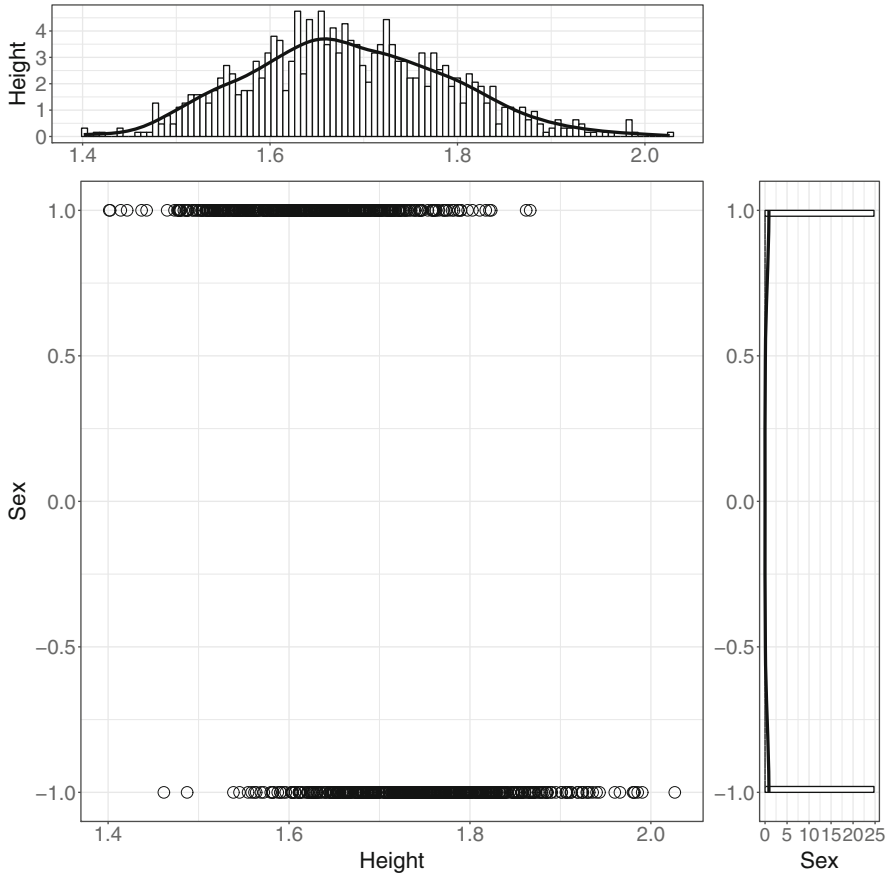


Fig. 1.13 Classifying sex according to heights. Top: a histogram and density of observed heights; right: the distribution of classes Male (-1) and Female (1); center: joint probability distribution (observe the need for plotting variables X, Y and the densities separately). A classifier would bisect the height axis into two classes

estimator for the expected risk $R(.)$. Equation (1.8) defines this concept in a reduced form when compared to Eq. (1.7).

$$R_{\text{emp}}(f) \rightarrow R(f), \text{ for } n \rightarrow \infty \tag{1.8}$$

In fact, the best classifier f_i is the one whose risk $R(.)$ is the lowest, i.e. $R(f_i) \leq R(f_j)$ for all j. However, there is no approach to compute $R(.)$ for real-world problems, so $R_{\text{emp}}(.)$ is computed instead for all classifiers $f_1, \ldots, f_k$, so that the one with the smallest empirical risk can be selected. As seen in the next chapter, this strategy is not that simple, once several other conditions rely on the described assumptions to make the empirical risk a good estimator for the (expected) risk.

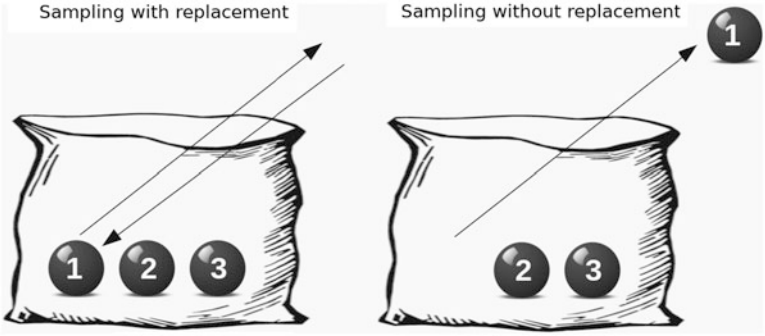


Fig. 1.14 Statistical independence illustrated through the sampling with and without replacement

Generalization The concept of generalization considers both the expected and the empirical risks, and it relies on the Law of Large Numbers which computes the absolute value of the difference between the average estimation of variable Ξ and its expected value (Eq. (1.9)). Such difference can be seen as a measure of distance between the estimator and the expected value. The less precise the estimator is, the greater such a distance is. The perfect estimator produces distance equals to zero.

$$\left|\frac{1}{n}\sum_{i=1}^{n}\Xi_i - E(\Xi)\right| \tag{1.9}$$

This concept is used in supervised learning to characterize how well the empirical risk $R_{\text{emp}}(.)$ estimates risk $R(.)$. A classifier f_i is said to generalize when $|R_{\text{emp}}(f_i) - R(f_i)| \approx 0$. Observe this does not ensure the best classifier is indeed f_i, it only informs us that the empirical risk of this classifier is a good estimator for its expected risk, given some sample from the joint probability distribution $P(X \times Y)$. Being the empirical risk a good estimator of $R(.)$, we can use $R_{\text{emp}}(.)$ as a proxy measure to select the best classifier.

Now, suppose all empirical risks of classifiers $f_1, \ldots, f_k$ provide a good enough generalization. Those risks can be used to sort classifiers out according to their losses, in form $R_{\text{emp}}(f_i) \leq R_{\text{emp}}(f_j) \leq \ldots$ in order to select the best one. From this perspective, we see how important is to build up some theoretical foundation to ensure good estimators.

In a practical point of view, people usually have a dataset with several examples which are divided into two sets: a training and a test set. The **training set** (seen examples) is used to infer classifiers $f_1, \ldots, f_k$ from. The **test set** contains unseen examples and is used to verify if the inferred classifiers are indeed good enough and which is the best. This simple evaluation technique relies on this concept of generalization and attempts to verify the risk of every classifier f_i, given unseen examples.

We can use this simple scenario to evaluate the generalization of any classifier f. Let a supervised algorithm infer f over the training set and consider the empirical risk $R_{\text{emp}}(f) = 0.01$. Then, f is used to classify examples from the test set,

producing a loss equals to 0.11. The absolute difference $|R_{\text{emp}}(f) - 0.11| = |0.01 - 0.11| = 0.1$ provides a measure of how good the risk estimation is. Note that, instead of computing $|R_{\text{emp}}(f) - R(f)|$, we calculated $|R_{\text{emp}}(f) - R'_{\text{emp}}(f)|$, given $R'_{\text{emp}}(f)$ is the empirical risk computed for an unseen finite sample (test set). As discussed in the next chapter, this last generalization in terms of two samples (one for training and another for testing) is an approximation for $|R_{\text{emp}}(f) - R(f)|$ after the Symmetrization lemma [17].

In order to assess the generalization of both classifiers illustrated in Fig. 1.4, suppose the linear classifier f has the following empirical risk $R_{\text{emp}}(f) = 0.12$ while the expected is $R(f) = 0.10$, and the polynomial classifier g has $R_{\text{emp}}(g) = 0$ but $R(g) = 0.9$. In this situation, the classifier providing the best generalization is f, once $|R_{\text{emp}}(f) - R(f)| = 0.02$ is minimal so that it is the best estimative for risk $R(.)$. By having a set of classifiers with a good enough generalization capacity, we can choose the one with the smallest empirical risk.

1.4 How a Supervised Algorithm Learns?

A supervised learning algorithm attempts to induce the best classifier f as possible so that $f : X \rightarrow Y$ provides the minimal risk $R(f)$. In this context, observe classifier f can also work as an **approximation or regression function** for the joint probability distribution $P(X \times Y)$, as illustrated in Fig. 1.15. So any supervised learning algorithm (e.g. Naive Bayes, SVM, Multilayer Perceptron, etc.) builds up some regression function, which is the same as inferring a classifier from a given training sample. Of course, if the sample collected from $P(X \times Y)$ is not sufficient, such regression will produce unacceptable losses.

Classifier f comes from a space of admissible functions $\mathscr{F}$. This space defines the **bias of the supervised algorithm**, that is, functions in $\mathscr{F}$ are often restricted to some types or families (e.g. linears, 2-order polynomials, etc.). As a consequence, if $P(X \times Y)$ has nonlinear characteristics but f has a linear bias, the supervised algorithm will produce an insufficient approximation that misses output classes more often than we wish. Although a classifier is in fact a regression function for the JPD, we can see it in another way which is by using hyperplanes or functions to "shatter" (or split) the input space into different regions. To better understand this approach, let us analyze a two-class dataset.

Let dataset $\mathscr{D} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ in which $x_i \in \mathbb{R}^2$ corresponds to the input variables (or input space) of example i, and $y_i \in \{+1, -1\}$ is its corresponding class. Consider the two input variables are temperature and humidity measurements, while the class defines whether a person plays $(+1)$ or does not play (-1) soccer under such conditions. Figure 1.16 illustrates this dataset, having $n = 100$ examples in which the circles represents label -1 (no), while the plus sign is associated with $+1$ (yes). By analyzing this plot, we have a good idea about how points are distributed in this bidimensional space.

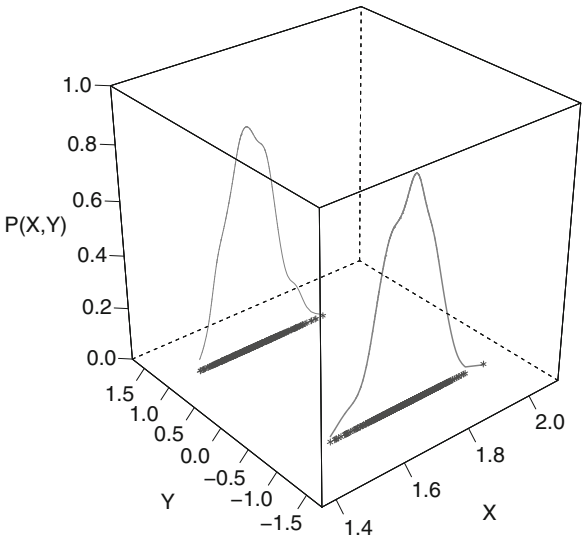


Fig. 1.15 Regression function produced to approximate the joint probability distribution illustrated in Fig. 1.13

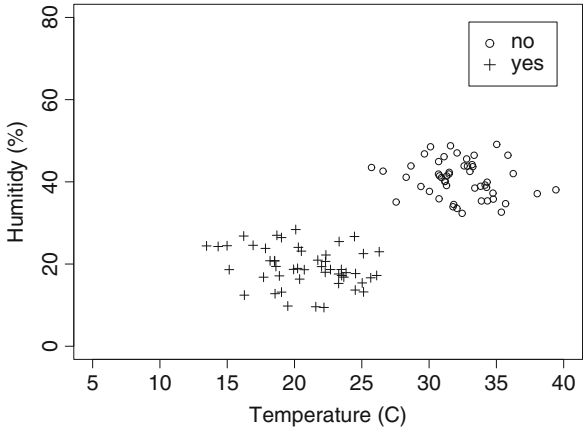


Fig. 1.16 Input space of examples containing two variables (temperature in Celsius and humidity in percentages) and two classes ($\{-1, +1\}$) for the problem of playing soccer

A supervised learning algorithm attempts to estimate $P(X \times Y)$ according to the available data sample $\mathscr{D}$ drawn from $P(X \times Y)$ in an independent and identical manner. As observed, there is a concentration of positive instances around a given mean point, and a concentration of negative instances around another mean. Let us assume that $P(X \times Y)$ is modeled by two Normal (or Gaussian) distributions as illustrated in Fig. 1.17, as the sample size $n \to \infty$.

Consider the JPD is known, so that one is able to employ a supervised learning algorithm whose space of admissible functions (or bias) $\mathscr{F}$ only contains Gaussian functions. Now consider our algorithm found a classifier f which is a composition of two Gaussian functions, each one centered at a different mean to represent each class. By assigning an identifier to each Gaussian (either -1 or $+1$), we can define

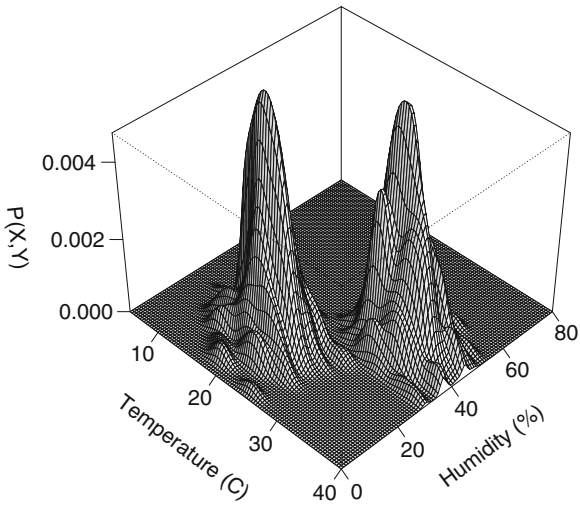


Fig. 1.17 Joint probability distribution $P(X \times Y)$ for the problem of playing soccer

the output labels for examples. All those steps make sense, but most well-known supervised learning algorithms are typically based on something simpler than that.

As an alternative to Gaussian functions, most of the supervised learning algorithms build **linear hyperplanes** to "shatter" the input space into regions, as shown in Fig. 1.18. Thus, every point on one side of the hyperplane will be classified as $+1$, while points on the other will be labeled as -1. Points lying on the hyperplane represent a tie, and, in principle, have no class assigned. Observe that this simpler approach was as effective (for this problem) as the Gaussians. In this case, we say the supervised learning algorithm has a linear bias instead.

The space containing all possible functions considered by a supervised algorithm, that is, the algorithm bias, is illustrated using a box as shown in Fig. 1.19. Inside such a box, the algorithm bias is circumscribed according to its restrictions. For example, the supervised algorithm called Perceptron [6] considers a single linear hyperplane to separate examples of binary problems, while the Multilayer Perceptron (MLP) may employ more than a single hyperplane to separate two or more classes. Therefore, the Perceptron bias is more restricted than the MLP.[3] As a consequence, the region illustrating the Perceptron bias is smaller than the one for MLP. We suggest the reader to associate a smaller region as a space containing less admissible functions to infer classifiers from.

We now illustrate the binary problem AND (examples defined in Table 1.5) in Fig. 1.20. Let $\mathbb{B}$ be the set of binary numbers. The input space contains examples $x_i \in \mathbb{B}^2$ and the corresponding output classes $y_i = \{0, 1\}$. This problem is used to discuss the Perceptron and MLP biases. By analyzing Fig. 1.20, we see a single

[3]When this algorithm employs more than a single hyperplane, otherwise it is the same as the Perceptron.

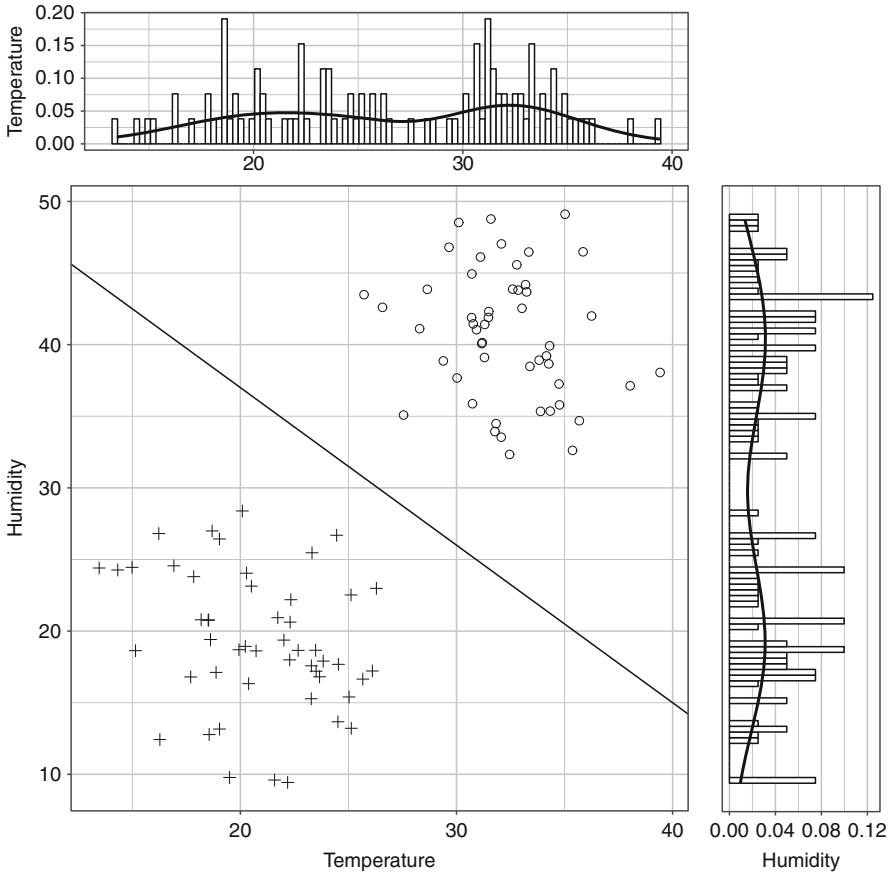


Fig. 1.18 Shattering of the input space of examples containing two variables (temperature and humidity) and two classes ($\{-1, +1\}$) for the problem of playing soccer

linear hyperplane is enough to provide the correct labels for `AND`, therefore the Perceptron algorithm satisfies such a task. However, what happens if we decide to employ the Perceptron to solve the problem `XOR` (Fig. 1.21)? Observe this new task requires a space with more admissible functions than a single hyperplane. This could be solved by using a Gaussian function as shown in Fig. 1.22, so it would provide an activation to characterize whether the class is 0 or 1. Another possible solution, considered by the Multilayer Perceptron (MLP), is to use more than a single linear hyperplane as shown in Fig. 1.23.

By using more than one hyperplane, MLP is capable of representing nonlinear behaviors. Comparing the Perceptron against the Multilayer Perceptron, we conclude the first has a linear while the second has a nonlinear bias if more hyperplanes are combined, as illustrated in Fig. 1.19. The Perceptron bias is more restricted (also referred to as a stronger bias), as it contains less functions (only linear ones) inside

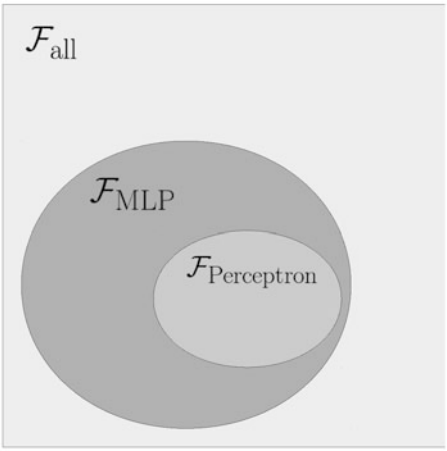


Fig. 1.19 Box illustrating the universe of admissible functions $\mathscr{F}_{all}$ for any supervised learning algorithm. Subspace $F_{\text{Perceptron}}$ corresponds to the Perceptron bias while F_{MLP} to the MLP bias (for MLP using two or more hyperplanes)

Table 1.5 Problem AND: Input space of examples and the corresponding classes

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

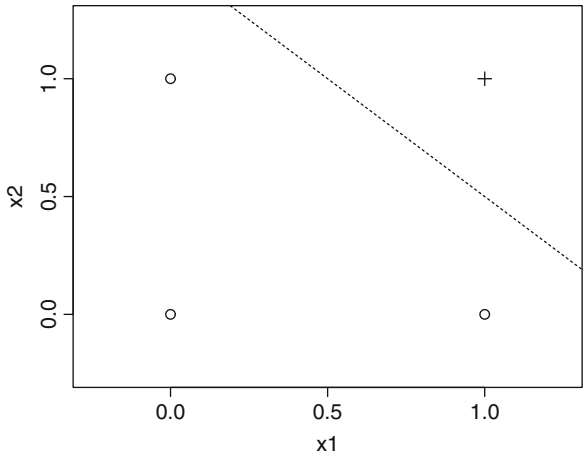


Fig. 1.20 Problem AND whose points correspond to examples and their shapes to classes: circle is associated to class 0, and cross to 1. Hyperplane illustrates a possible classifier

such space region, while the MLP bias is less restricted (also referred to as weaker), containing nonlinear functions. This is the case if and only if MLP considers two or more linear hyperplanes, otherwise it will have the same bias as the Perceptron.

All this analysis supports the study of the **Bias-Variance Dilemma** (discussed in Sects. 1.3 and 1.4). By having a smaller subspace (a.k.a. stronger bias) of admissible functions, the Perceptron has less variance associated to the convergence to its best

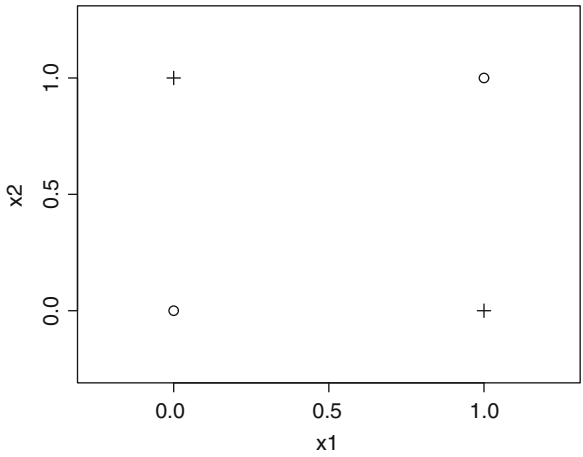


Fig. 1.21 Illustration of the problem XOR in which points correspond to examples and their shapes to classes: circle is associated to class 0, and cross to 1

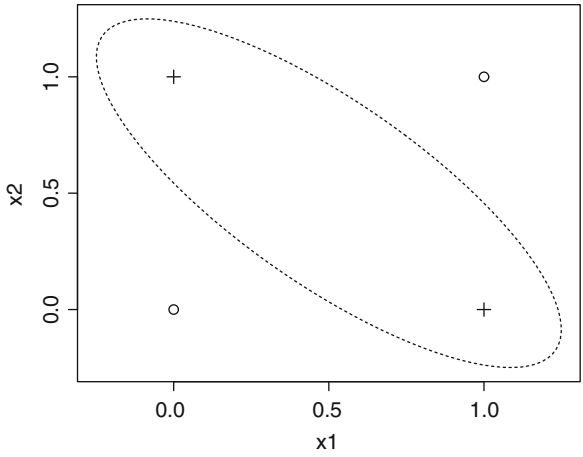


Fig. 1.22 Problem XOR approached with a Gaussian function

classifier $f^{(b)}_{\text{Perceptron}} \in F_{\text{Perceptron}}$. On the other hand, MLP has a greater variance, what makes it require more samples to converge to its best classifier $f^{(b)}_{\text{MLP}} \in F_{\text{MLP}}$.

Back to problem XOR, let us suppose the Perceptron and MLP were trained to infer their best classifiers $f^{(b)}_{\text{Perceptron}}$ and $f^{(b)}_{\text{MLP}}$, respectively. In such scenario, $f^{(b)}_{\text{Perceptron}}$ is not enough to solve this task, producing a greater risk. On the other hand, MLP would provide a solution using two hyperplanes as shown in Fig. 1.23. How does that affect the biases of both algorithms? Figure 1.24 illustrates a possible location for those best classifiers. Observe the Perceptron would never converge to $f^{(b)}_{\text{MLP}}$, because it is outside of its space of admissible functions.

For the problem AND, a single hyperplane is enough (Fig. 1.20), consequently the two-hyperplane MLP and the Perceptron represent the same solution as illustrated in Fig. 1.25. Note the biases of both algorithms and the location of the best classifier $f^{(b)}$, which ends up to be the same for both classification algorithms.

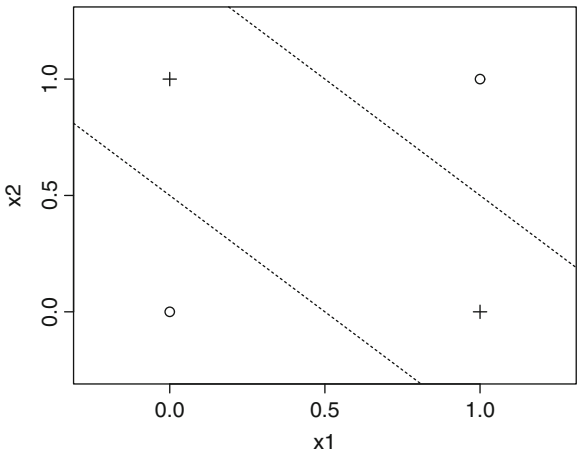


Fig. 1.23 Problem XOR approached with the Multilayer Perceptron (MLP) using two linear hyperplanes

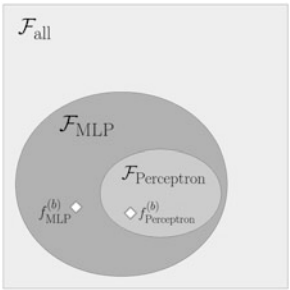


Fig. 1.24 The universe of functions $\mathscr{F}_{all}$ which contains the Perceptron and MLP biases. The best Perceptron classifier is different from the best for MLP

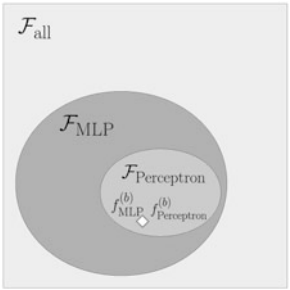


Fig. 1.25 The universe of functions $\mathscr{F}_{all}$ contains all biases, including a more restricted subspace of functions related to the Perceptron and MLP biases. For the problem AND, the best Perceptron classifier is the same as for MLP

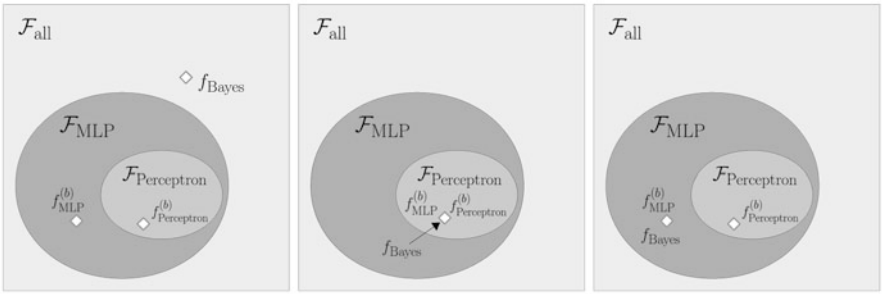


Fig. 1.26 The best of all classifiers f_{Bayes} and the biases of MLP and Perceptron. (**a**) f_{Bayes} is not contained in the biases of MLP and Perceptron; (**b**) f_{Bayes} is contained in Perceptron, and consequently in MLP bias; (**c**) f_{Bayes} is only inside the MLP bias

This second situation is very interesting. Both algorithms will attempt to converge to $f^{(b)} = f^{(b)}_{\text{Perceptron}} = f^{(b)}_{\text{MLP}}$. Which one should we use (the Perceptron or the MLP) to tackle such a task? Both can be used, but the smaller variance helps the Perceptron converge faster to $f^{(b)}$, so it would be undoubtedly a better choice.

We conclude the Perceptron bias is enough for the problem `AND`, while it is not sufficient for `XOR`. The reader is invited to analyze the biases of other supervised algorithms. Whenever the algorithm bias is not sufficient to induce a suitable classifier for a given task, we should allow more admissible functions. That is why we represent such weaker bias with a greater region as shown in Fig. 1.19. Now we recall that if MLP is parametrized to use a single hyperplane, it will have the same bias as the Perceptron. By using more hyperplanes, we make the space larger, i.e. the bias weaker.

After introducing the concept of algorithm bias, let us introduce the Bayes classifier f_{Bayes}, which is the best function in $\mathscr{F}$. As illustrated in Fig. 1.26, an algorithm may converge to it only if such function is inside its bias. Classifier f_{Bayes} is the best approximation for the joint probability distribution $P(X \times Y)$, thus reducing the expected risk $R(f_{\text{Bayes}})$ as much as possible. Depending on the data, that might not be exactly zero, because classes may be overlapped due to nondeterminism or noise (e.g. the problem involving a die).

Figure 1.26a depicts the situation in which the Perceptron and MLP do not contain f_{Bayes}, so they would converge to $f^{(b)}_{\text{Perceptron}}$ and $f^{(b)}_{\text{MLP}}$, respectively. In Fig. 1.26b, both algorithms can converge to f_{Bayes}, so $f_{Bayes} = f^{(b)}_{\text{Perceptron}} = f^{(b)}_{\text{MLP}}$; and finally, in Fig. 1.26c, the MLP bias contains f_{Bayes}, so $f_{\text{Bayes}} = f^{(b)}_{\text{MLP}}$ but $f^{(b)}_{\text{Perceptron}} \neq f_{\text{Bayes}}$, thus MLP may converge to such best function as more training examples are provided.

It is also worth mentioning that a given algorithm bias may contain more than one classifier providing the same solution, i.e. more than a single f_{Bayes}, $f^{(b)}_{\text{Perceptron}}$ and $f^{(b)}_{\text{MLP}}$ in $\mathscr{F}_{all}$. Observe we could slightly move the hyperplanes from Figs. 1.20 and 1.23 and still have the same classification results.

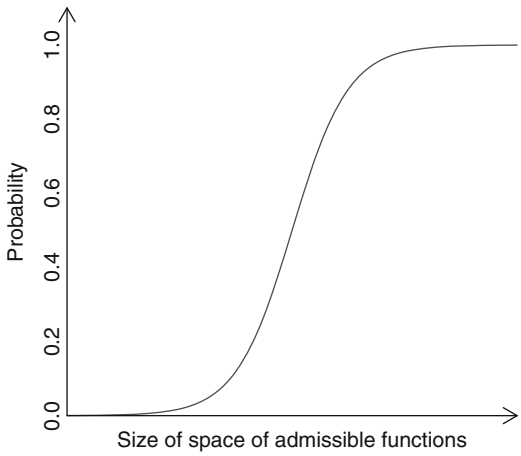


Fig. 1.27 When the space of admissible functions $\mathscr{F}_{\text{algorithm}}$ grows, its probability of containing the memory-based classifier increases. This is just an illustration: this probability function is not necessarily a sigmoid such as depicted

Algorithm Bias and the Memory-Based Classifier The algorithm bias is a central concept for the Statistical Learning Theory. For example, consider an algorithm has a bias containing a **memory-based classifier** f_m capable of producing an empirical risk $R_{\text{emp}}(f_m) = 0$ by memorizing all training examples, but failing for unseen examples. In detail, in a binary classification problem, for any unseen example received as input, this algorithm produces a random output: let 50% of examples lying in class $+1$ while the other half in -1. Obviously, in average, it will output $+1$ correctly for half of unseen examples, misclassifying the other half. Assessing f_m with the $0-1$-loss function, we observe $R(f_m) = 0.5$, as half of unseen examples will be associated to the wrong class. As a consequence, f_m does not provide a good generalization, i.e., $|R_{\text{emp}}(f_m) - R(f_m)| = 0.5$, once it will never approach zero as the sample size n tends to infinity. Therefore, any algorithm capable of admitting the memory-based classifier may eventually converge to f_m and, thus, never generalize knowledge.

This is one of the most important issues studied by the SLT: an algorithm should never have the memory-based classifier inside its bias $\mathscr{F}_{\text{algorithm}}$, otherwise no learning is guaranteed, but solely the memorization of training examples. That represents a challenge: by setting a bias with more admissible functions to address a given problem, space $\mathscr{F}_{\text{algorithm}}$ will contain more classifiers to choose from, increasing the probability of containing the memory-based classifier (as shown in Fig. 1.27).

From Fig. 1.27, we conclude some bias is necessary to ensure learning, once a larger space of admissible functions may contain the memory-based classifier. For example, consider the data depicted in Fig. 1.28 and assume the supervised algorithm is capable of inferring any polynomial function. Therefore, it could converge to a perfect nonlinear function with $R_{\text{emp}}(f) = 0$, i.e., it does not make mistakes in the training sample, however it could fail miserably on unseen examples, an unacceptable result.

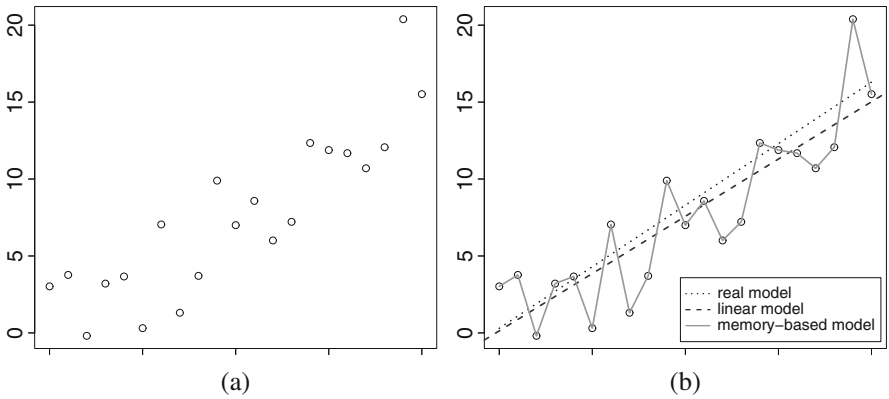


Fig. 1.28 The dotted line shown is the real model, from which noisy sample data (circles) were obtained. A regression is approached by two algorithms with different biases: the first includes only linear functions, while the second includes a high-order polynomial function, resulting in a memory-based model. (**a**) Noisy sample of points, (**b**) different models to fit the points

The concepts introduced so far are detailed, in the point of view of SLT, in Chap. 2. However, before that, it is important to have tools to instantiate such concepts in practical scenarios. The Perceptron and the Multilayer Perceptron were selected since those algorithms provide excellent case studies for supervised learning.

1.5 Illustrating the Supervised Learning

This section introduces the Perceptron and the Multilayer Perceptron, including their formulations and implementations using the R language [10].

1.5.1 The Perceptron

As discussed in Sect. 1.4, the Perceptron bias is linear and therefore it contains less functions inside space $\mathscr{F}_{\text{Perceptron}}$ than any MLP using more than a single hyperplane. As a consequence, this algorithm is adequate to model linearly separable problems such as `AND` and `OR`. On the other hand, the Perceptron does not suit more complex problems (even toy problems such as `XOR`).

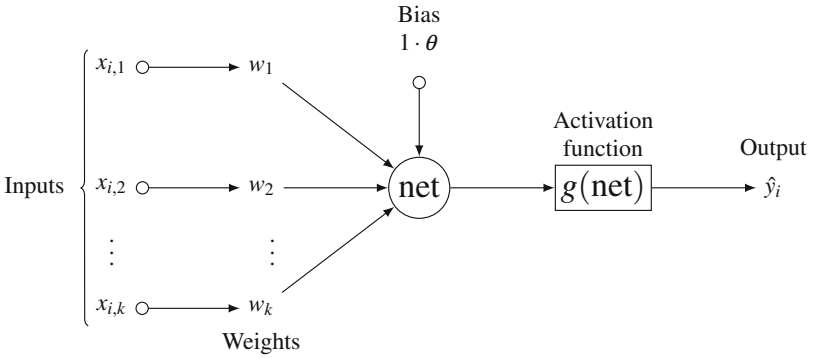


Fig. 1.29 The Perceptron: input $x_i \in \mathbb{R}^k$, k connection weights w_j and the bias term θ. This neuron computes net as a sum of inputs multiplied by weights plus θ. The real-valued output is given by $\hat{y}_i = g(\text{net})$

The Perceptron algorithm was proposed by Rosenblatt [11] in 1957. It considers an artificial neuron or unit[4] that receives an input example x_i to produce an output class y_i, as illustrated in Fig. 1.29. Such input example $x_i \in \mathbb{R}^k$, i.e. it contains a list of indexed variables $j = 1, \ldots, k$.

The neuron multiplies every input value $x_{i,j}$ by a weight w_j as well as constant 1 by the bias term[5] denoted by θ. As a next step, this algorithm computes a term $\text{net} = \sum_{j=1}^{k} x_{i,j} w_j + \theta$. Note this represents a linear combination of $x_{i,j}$, for which θ is the interception in relation to the axes defined by $x_{i,j} \forall i$. Next, the Perceptron applies a heaviside (or step) function $g(\text{net})$ (Eq. (1.10)), in which ϵ is a user-defined parameter, producing the output class $\hat{y}_i$ for example x_i.

$$g(\text{net}) = \begin{cases} 1, & \text{iff net} > \epsilon \\ 0, & \text{otherwise} \end{cases} \tag{1.10}$$

In order to better understand the Perceptron algorithm, we approach a problem with a single variable $x_{i,1}, \forall i$ provided as input. Let every real input value $0 \leq x_{i,1} \leq 0.5$ be assigned to class 0, while $0.5 < x_{i,1} \leq 1.0$ is assigned to class 1, such as the training sample provided in Table 1.6. Figure 1.30 shows the expected classes y_i for every possible input value in range [0, 1].

In this case, the Perceptron adapts parameters w_1 (weight) and θ (bias) to build up a linear function $\text{net} = x_{i,1} w_1 + \theta$, in form of $ax + b$, having $x = x_{i,1}$ and $b = \theta$, as shown in Fig. 1.31. If we set $\epsilon = 0$ and apply every value of net into Eq. (1.10), we obtain the expected outputs as shown in the same figure. The Perceptron learning process consists in searching for adequate weights w_j and bias θ such that $ax + b$ has the correct linear shape.

[4]The term neuron is used due to the biological neuron motivation.

[5]The reader may not confuse this bias term with the Bias-Variance Dilemma. The bias term θ is simply a space interception value.

Table 1.6 The Perceptron: training sample for a single-variable problem

Input variable	Output class
0.0	0
0.1	0
0.2	0
0.3	0
0.4	0
0.5	0
0.6	1
0.7	1
0.8	1
0.9	1
1.0	1

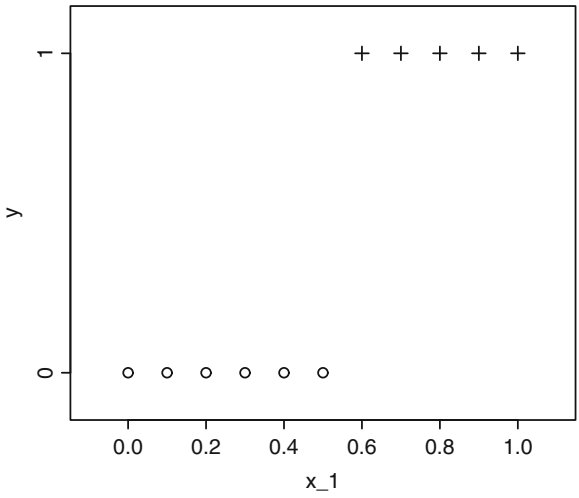


Fig. 1.30 The Perceptron on a simple problem: the expected classes for the range of the input variable

To adapt weights and the bias term, the Perceptron considers the empirical risk $R_{\text{emp}}(f_{\text{candidate}})$, i.e., the error considering the training set for every candidate classifier. The squared-error is used as the loss function to compute the empirical risk (as defined in Eq. (1.11)), because it provides a convex function when the classes involved in the problem are linearly separable, i.e., separable using a linear hyperplane. We discuss about this issue later on. For now, let us employ this function to find the solution.

$$\ell_{\text{squared}}(x_i, y_i, g(x_i)) = (y_i - g(x_i))^2 \tag{1.11}$$

At this point, it may be clear that we want to minimize error (or risk) considering the training sample. In that sense, $g(.)$ should produce the smallest error when analyzing an input example x_i, as defined in Eq. (1.12). Figure 1.32 illustrates this squared-error function in terms of the free variables, i.e., variables adapted by

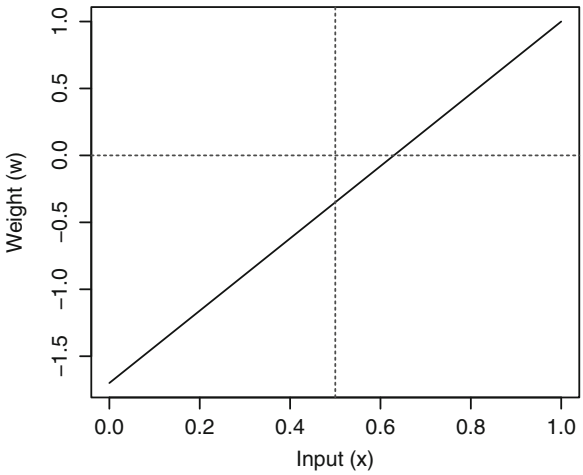


Fig. 1.31 The Perceptron on a simple problem: linear approximation given net

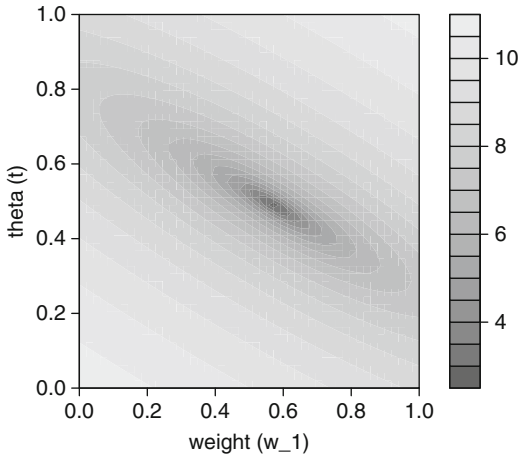


Fig. 1.32 Squared-error loss function computed for some training sample with n observations. The x-axis corresponds to parameter w_1, while the y-axis to bias term θ

Perceptron while finding a good linear approximation function to solve this problem. The Perceptron aims to converge to the minimum of function E^2.

$$E^2 = \sum_{i=1}^{n} \ell_{\text{squared}}(x_i, y_i, g(x_i)) = \sum_{i=1}^{n} (y_i - g(x_i))^2 \tag{1.12}$$

Let us assume this squared-error function provides a perfect paraboloid as shown in Fig. 1.32, what might not occur once it depends on the available training examples and also on the loss function. Consider also initial values $w_1 = -1$ and $\theta = 1$. We would have a linear function net as shown in Fig. 1.33, when analyzing the input and output spaces X and Y, respectively. After applying the heaviside function $g(\text{net})$ (Eq. (1.10)) with $\epsilon = 0.5$, we would obtain a plot as shown in Fig. 1.34.

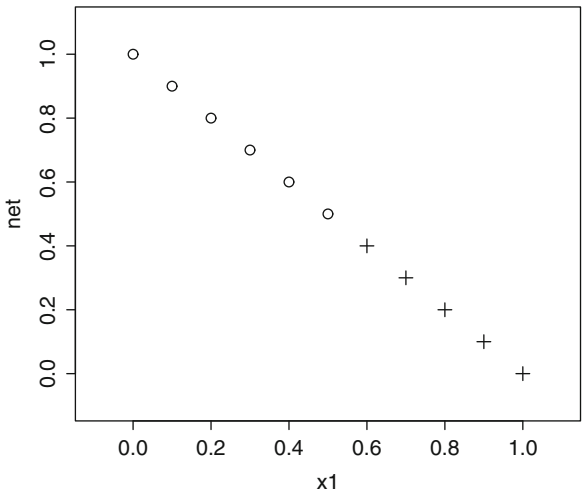


Fig. 1.33 The Perceptron: input and output spaces X and Y for function net

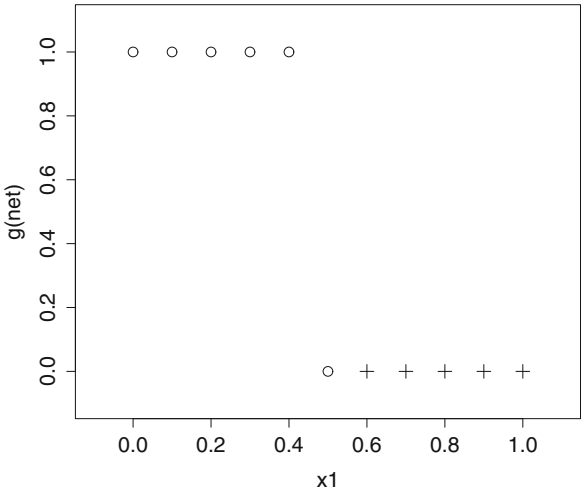


Fig. 1.34 The Perceptron: input and output spaces X and Y for function $g(\text{net})$

For the sake of simplicity, observe $w_1 = -1$ as a projected point touching the squared error function illustrated in Fig. 1.32. Now, let the Perceptron adapt w_1 so it converges to a good enough function $g(\text{net})$. The gradient information on the projected point indicates the direction, i.e. how to modify w_1, in order to approach the solution with the smallest risk as possible. Note the same happens for variable θ.

As there is a negative tendency at point $w_1 = -1$, given by the tangent slope, the Perceptron should increase the value of w_1. This is exactly what is defined by the Gradient Descent (GD) method as formalized in Eq. (1.13), in which $\eta \in (0, \infty)$ is a user-defined parameter, a.k.a. gradient step.

In the case of a negative derivative, $\frac{\partial E^2}{\partial w_1} < 0$, its multiplication by $-\eta$ results in an increment of the current $w_1(t)$. Of course, if η is too small, this increment

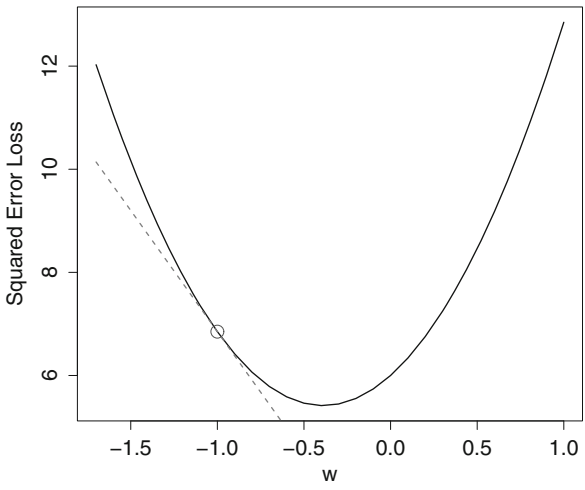


Fig. 1.35 Squared-error function E^2 in terms of the free variable w_1

will be also small, taking too long to converge to the minimal squared error. On the other hand, if η is too great, then $w_1(t+1)$ may cross the minimum and reach the other side, thus the derivative $\frac{\partial E^2}{\partial w_1}$ will be positive in the next step (Fig. 1.35). Note η influences the convergence rate and, in some cases, can even cause divergence. Equation (1.14) defines the GD method for the second free variable θ.

$$w_1(t+1) = w_1(t) - \eta \frac{\partial E^2}{\partial w_1} \tag{1.13}$$

$$\theta(t+1) = \theta(t) - \eta \frac{\partial E^2}{\partial \theta} \tag{1.14}$$

Now, the formulation to adapt the Perceptron variables is presented. As first requirement, terms $\frac{\partial E^2}{\partial w_1}$ and $\frac{\partial E^2}{\partial \theta}$ must be computed, given $E^2 = \sum_{i=1}^{n}(y_i - f(x_i))^2$, and $f(x_i)$ written as:

$$f(x_i) = g(\text{net}_i) = g\left(\sum_{j=1}^{k} x_{i,j} w_j + \theta\right).$$

However, as $g(.)$ is the heaviside function, we could not directly differentiate it. As an approximation, Rosenblatt relaxed such differentiation by considering:

$$f(x_i) = \text{net}_i = \sum_{j=1}^{k} x_{i,j} w_j + \theta,$$

then, the squared-error function is given by:

$$E^2 = \sum_{i=1}^{n}(y_i - f(x_i))^2 = \sum_{i=1}^{n}\left(y_i - \left[\sum_{j=1}^{k} x_{i,j}w_j + \theta\right]\right)^2,$$

so, $\frac{\partial E^2}{\partial w_1}$ and $\frac{\partial E^2}{\partial \theta}$ are found via the chain rule for derivatives as follows:

$$\frac{\partial E^2}{\partial w_1} = 2\sum_{i=1}^{n}(y_i f - f(x_i))\frac{\partial\left[y_i - f(x_i)\right]}{\partial w_1},$$

and:

$$\frac{\partial E^2}{\partial \theta} = 2\sum_{i=1}^{n}(y_i - f(x_i))\frac{\partial\left[y_i - f(x_i)\right]}{\partial \theta}.$$

Recall y_i is the expected output class for example i (i.e., a label defined by some specialist), thus when differentiating it in terms of either w_1 or θ, y_i is disconsidered. Differentiating $\frac{\partial - f(x_i)}{\partial w_1}$ and $\frac{\partial - f(x_i)}{\partial \theta}$:

$$\frac{\partial - f(x_i)}{\partial w_1} = -\frac{\partial\left[x_{i,1}w_1 + \theta\right]}{\partial w_1} = -x_{i,1},$$

and:

$$\frac{\partial - f(x_i)}{\partial \theta} = -\frac{\partial\left[x_{i,1}w_1 + \theta\right]}{\partial \theta} = -1.$$

The Gradient Descent method is formulated as follows:

$$\begin{aligned} w_1(t+1) &= w_1(t) - \eta\frac{\partial E^2}{\partial w_1} \\ &= w_1(t) - \eta\, 2\sum_{i=1}^{n}(y_i - f(x_i))(-x_{i,1}), \end{aligned}$$

and:

$$\begin{aligned} \theta(t+1) &= \theta(t) - \eta\frac{\partial E^2}{\partial \theta} \\ &= \theta(t) - \eta\, 2\sum_{i=1}^{n}(y_i - f(x_i))(-1), \end{aligned}$$

which, in open-form, is written as:

$$w_1(t+1) = w_1(t) - \eta \frac{\partial E^2}{\partial w_1}$$
$$= w_1(t) - \eta\, 2 \sum_{i=1}^{n} (y_i - \left[x_{i,1} w_1 + \theta\right])(-x_{i,1}), \tag{1.15}$$

and, finally:

$$\theta(t+1) = \theta(t) - \eta \frac{\partial E^2}{\partial \theta}$$
$$= \theta(t) - \eta\, 2 \sum_{i=1}^{n} (y_i - \left[x_{i,1} w_1 + \theta\right])(-1). \tag{1.16}$$

Observe constant 2 is irrelevant due to another constant η defining the gradient step. Those two last equations correspond to the Perceptron learning process, as implemented in Listing 1.2. Instead of adapting w_1 and θ after analyzing all input examples, as the summation used in Eqs. (1.15) and (1.16), our code performs the GD method on an example basis.

The `R` functions in Listing 1.2 implement a simple classification task in which a single variable represents the input, i.e. $x_i \in \mathbb{R}$, producing output classes in set $\{0, 1\}$. This listing contains: (1) the heaviside function (lines `4-9`); (2) the Perceptron training function, which returns the model, i.e. the learned parameters w_1 and θ (lines `14-87`); (3) a function responsible to perform the classification of new instances, given the trained model (lines `92-115`); and, finally, (4) a demo function with training and test sets (lines `127-169`).

Listing 1.2 The Perceptron—implementation of the simplest classification task ("perceptron.r")

```
1  # Source code file: "perceptron.r"
2
3  # Heaviside function with a default epsilon
4  g <- function(net, epsilon=0.5) {
5      if (net > epsilon) {
6          return (1)
7      } else {
8          return (0)
9      }
10 }
11
12 # This is the function to train the Perceptron
13 # Observe eta and threshold assume default values
14 perceptron.train <- function(train.table, eta=0.1,
15                                          threshold=1e-2) {
16
17     # Number of input variables
```

```
18  nVars = ncol(train.table)-1
19
20  cat("Randomizing weights and theta in range [-0.5, 0.5]\n
        ")
21
22  # Randomizing weights
23  weights = runif(min=-0.5, max=0.5, n=nVars)
24
25  # Randomizing theta
26  theta = runif(min=-0.5, max=0.5, n=1)
27
28  # This sum of squared errors will accumulate all errors
29  # occurring along training iterations. When this error is
30  # below a given threshold, learning stops.
31  sumSquaredError = 2*threshold
32
33  # Learning iterations
34  while (sumSquaredError > threshold) {
35
36     # Initializing the sum of squared errors as zero
37     # to start counting and later evaluate the total
38     # loss for this dataset in train.table
39     sumSquaredError = 0
40
41     # Iterate along all rows (examples) contained in
42     # train.table
43     for (i in 1:nrow(train.table)) {
44
45        # Example x_i
46        x_i = train.table[i, 1:nVars]
47
48        # Expected output class
49        # Observe the last column of this table
50        # contains the output class
51        y_i = train.table[i, ncol(train.table)]
52
53        # Now the Perceptron produces the output
54        # class using the current values for
55        # weights and theta, then it applies the
56        # heaviside function
57        hat_y_i = g(x_i %*% weights + theta)
58
59        # This is the error, referred to as (y_i - g(x_i))
60        # in the Perceptron formulation
61        Error = y_i - hat_y_i
62
63        # As part of the Gradient Descent method, we here
64        # compute the partial derivative of the Squared
              Error
65        # for the current example i in terms of weights and
66        # theta. Observe constant 2 is not necessary, once
              we
67        # can set eta using the value we desire
68        dE2_dw1 = 2 * Error * -x_i
```

```
69            dE2_dtheta = 2 * Error * -1
70
71            # This is the Gradient Descent method to adapt
72            # weights and theta as defined in the formulation
73            weights = weights - eta * dE2_dw1
74            theta = theta - eta * dE2_dtheta
75
76            # Accumulating the squared error to define
77            # the stop criterion
78            sumSquaredError = sumSquaredError + Error^2
79        }
80
81        cat("Sum of squared errors = ", sumSquaredError, "\n")
82    }
83
84    # Returning weights and theta, once they represent
85    # the solution
86    ret = list()
87    ret$weights = weights
88    ret$theta = theta
89
90    return (ret)
91 }
92
93 # This is the function to execute the Perceptron
94 # over unseen data (new examples)
95 perceptron.test <- function(test.table, weights, theta) {
96
97    # Here we print out the expected class (yi) followed by
          the
98    # obtained one (hat_yi) when considering weights and
          theta.
99    # Of course, function perceptron.train should be called
100   # previously to find the values for weights and theta
101   cat("#yi\that_yi\n")
102
103   # Number of input variables
104   nVars = ncol(test.table)-1
105
106   # For every row in the test.table
107   for (i in 1:nrow(test.table)) {
108
109      # Example i
110      x_i = test.table[i, 1:nVars]
111
112      # Expected class for example i
113      y_i = test.table[i, ncol(test.table)]
114
115      # Output class produced by the Perceptron
116      hat_y_i = g(x_i %*% weights + theta)
117
118      cat(y_i, "\t", hat_y_i, "\n")
119   }
120 }
```

```
121
122 # This is an example of learning the simplest problem
123 # To run this example:
124 #    1) Open the R Statistical Software
125 #    2) source("perceptron.r")
126 #    3) perceptron.run.simple()
127 #
128 perceptron.run.simple <- function() {
129
130    # This is a table with training examples
131    train.table = matrix(c(0.0, 0,
132                           0.1, 0,
133                           0.2, 0,
134                           0.3, 0,
135                           0.4, 0,
136                           0.5, 0,
137                           0.6, 1,
138                           0.7, 1,
139                           0.8, 1,
140                           0.9, 1,
141                           1.0, 1),
142                         nrow=11,
143                         ncol=2,
144                         byrow=TRUE)
145
146    # This is a table with test examples.
147    # The last column only shows the expected
148    # output and it is not used in the testing stage
149    test.table = matrix(c(0.05, 0,
150                          0.15, 0,
151                          0.25, 0,
152                          0.35, 0,
153                          0.45, 0,
154                          0.55, 1,
155                          0.65, 1,
156                          0.75, 1,
157                          0.85, 1,
158                          0.95, 1),
159                        nrow=10,
160                        ncol=2,
161                        byrow=TRUE)
162
163    # Training the Perceptron to find weights and theta
164    training.result = perceptron.train(train.table)
165
166    # Testing the Perceptron with the weights and theta found
167    perceptron.test(test.table, training.result$weights,
168                    training.result$theta)
169
170    return (training.result)
171 }
```

To run this example, the reader must load the source code in the R Statistical Software. Afterwards, (s)he may execute function `perceptron.run.simple()`.[6] Notice the expected and obtained output classes may contain errors when the input variable $x_{i,1}$ is nearby the transition value 0.5, as it is the point splitting up labels 0 and 1.

By running `perceptron.run.simple()`, the textual output should be similar to the one illustrated in Listing 1.3. At first, w_1 and θ are randomized, then the Perceptron starts iterating on training examples until the sum of squared errors (for the entire set) converges to zero. The expected y_i and the obtained $\hat{y}_i$ classes for every test example are listed. A careful reader may note that one of the output classes $\hat{y}_i$ is not correct in Listing 1.3 because: (1) the hyperplane was fitted according to the training set; and (2) the test set contains unseen examples whose variations were not seen during training.

Listing 1.3 Text output produced by function perceptron.run.simple()

```
Randomizing weights and theta in range [-0.5, 0.5]...
Accumulated sum of squared errors =  2
Accumulated sum of squared errors =  2
Accumulated sum of squared errors =  2
Accumulated sum of squared errors =  2
Accumulated sum of squared errors =  1
Accumulated sum of squared errors =  0
#yi      hat_yi
0         0
0         0
0         0
0         0
0         0
1         0
1         1
1         1
1         1
1         1
$weights
[1] 0.3979496

$theta
[1] 0.2652314
```

In order to visualize how the Perceptron separates the input space, see function `perceptron.simple.hyperplane.plot(w,t)` in Listing 1.4 which plots the resulting hyperplane. It requires weight w_1 and theta θ obtained after running function `perceptron.run.simple()` from Listing 1.4. An example of hyperplane is illustrated in Fig. 1.36 (note the effect of the heaviside function, which discretizes output values as either 0 or 1).

[6] We suggest to execute this function several times, in order to see the effects of using different starting random values for weight w_1 and θ.

Listing 1.4 The Perceptron—function for plotting the hyperplane found for the simplest classification task

```
source("perceptron.r")

# This function plots the hyperplane found for this simplest
# problem which considers a single input variable.
# Variables range.start and range.end define the interval of
# values for the single input variable composing the problem
    .
# This simple problem has a single variable composing every
# example i, which is x_i,1
perceptron.simple.hyperplane.plot <- function(weight, theta,
                                              range.start=0,
                                              range.end=1) {

   # Number of variables is 1
   nVars = 1

   # We will now define the same range for the input
       variable.
   # This range will contain 100 discretized values
   range_of_every_input_variable =
              seq(range.start, range.end, length=100)
   x_1 = range_of_every_input_variable

   # Computing net for every input value of variable x_i,1
   all_nets = cbind(x_1, 1) %*% c(weight, theta)

   # This variable all_nets contains all net values for all
   # values assumed by variable x_1. Variable hat_y will
   # contain the Perceptron outputs after applying the
   # heaviside function
   hat_y = rep(0, length(all_nets))
   for (i in 1:length(all_nets)) {
      hat_y[i] = g(all_nets[i])
   }

   # Variable hyperplane will contain two columns, the first
   # corresponds to the input value of x_i,1 and the second
   # to the class produced by the Perceptron
   hyperplane = cbind(x_1, hat_y)

   # Plotting the hyperplane found by the Perceptron
   plot(hyperplane)

   return (hyperplane)
}
```

In order to analyze the obtained hyperplane, the heaviside function is omitted so that the output is only net(.) in function `perceptron.simple.hyperplane.plot.without.g()`. The Perceptron was executed five times using function `perceptron.run.simple()`, recording the corresponding weights and thetas

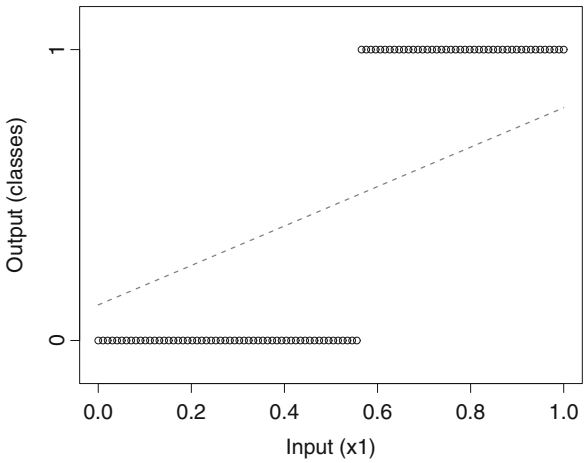


Fig. 1.36 Illustration of the hyperplane (dashed line) produced by the Perceptron, with outputs $\hat{y}_i$ along different input values of $x_{i,1}$

(see Listing 1.5). Then, each pair weight and theta were used to compute five different functions net(.) using function `perceptron.simple.hyperplane.plot.without.g()`.

Listing 1.5 The Perceptron—plotting the hyperplane in terms of function net

```
source("perceptron.r")

# This function plots the hyperplane found for this simplest
# problem which considers a single input variable and
# function net only.
# Variables range.start and range.end define the interval of
# values for the single input variable composing the problem
    .
# This simple problem has a single variable composing every
# example i, which is x_i,1
perceptron.simple.hyperplane.plot.without.g <-
              function(weight, theta, range.start=0,
                                      range.end=1) {

   # Number of variables is 1
   nVars = 1

   # We will now define the same range for the input
       variable.
   # This range will contain 100 discretized values
   range_of_every_input_variable =
          seq(range.start, range.end, length=100)
   x_1 = range_of_every_input_variable

   # Computing net for every input value of variable x_i,1
   all_nets = cbind(x_1, 1) %*% c(weight, theta)
```

```
    # This variable all_nets contains all net values for
        every
    # value assumed by variable x_1. Variable hat_y will
        contain
    # Perceptron outputs before applying the heaviside
        function
    hat_y = rep(0, length(all_nets))
    for (i in 1:length(all_nets)) {
       hat_y[i] = all_nets[i] # No heaviside function g(net)
    }

    # Variable hyperplane will contain two columns, the first
    # corresponds to the input value of x_i,1 and the second
    # to the class produced by the Perceptron
    hyperplane = cbind(x_1, hat_y)

    # Plotting the hyperplane found by Perceptron in terms of
    # function net
    plot(hyperplane)

    return (hyperplane)
}
```

Listing 1.6 implements the repeated training executions. Figure 1.37 exemplifies five net(.) functions. Each training stage is executed after calling `perceptron.run.simple()`, which invokes `perceptron.train()` and, consequently, the heaviside function $g(\text{net})$, using the threshold $\epsilon = 0.5$. By omitting the heaviside function in Fig. 1.37, one can see the effects produced by parameter ϵ, as defined in Eq. (1.10).

Figure 1.37 makes evident that parameter ϵ is responsible for defining the same central point for hyperplanes in terms of the y-axis, so their slopes change but they preserve the same pivot. Even for different positive slopes, the classification results are the same after applying $g(\text{net})$. Also observe that training with different values of ϵ make hyperplanes assume different pivots as shown in Fig. 1.38 for $\epsilon = \{0.25, 0.50, 0.75\}$. In fact, any other value of ϵ would not affect the final result, once Eq. (1.10) simply uses it as a heaviside threshold, to separate label 0 from 1.

Listing 1.6 Details about the several training executions

```
source("perceptron-hyperplane-without-g.r")

# This function is used to run the training stage for the
# simplest classification task. Each training will produce
# a different pair of weight and theta, which are then used
# to plot function net
run.several.times <- function(times=5) {

    # Saving the results for each function net
    net.functions = list()

    # For each execution
```

```
    for (i in 1:times) {
        # Call training
        training.execution = perceptron.run.simple()

        # Obtaining function net
        net.functions[[i]] =
              perceptron.simple.hyperplane.plot.without.g(
                 training.execution$weight,
                 training.execution$theta)
    }

    # Plotting
    plot(net.functions[[1]], col=1)
    for (i in 2:times) {
        points(net.functions[[i]], col=i)results
    }
}
```

We have already advanced in several aspects of the Perceptron algorithm, but illustrations are still needed to understand the error and the squared-error functions. In Listing 1.7, function `perceptron.simple.error()` produces the error, i.e., the difference between the expected and obtained classes $y_i - \hat{y}_i$. It also plots such an error in terms of weight and theta, as shown in Fig. 1.39. Observe weight and theta variations imply in a linear surface of errors.

By using the gradient to modify the free variables so that they minimize the error, the solution would tend to minus infinity because the function has an undefined minimal point, therefore there is no stop condition. One might even think about applying the Newton-Raphson method [18] to find successive approximations for the roots of this real-valued error function, but positive and negative errors might cancel out each other. Consequently, the roots may not represent the zero error as expected.

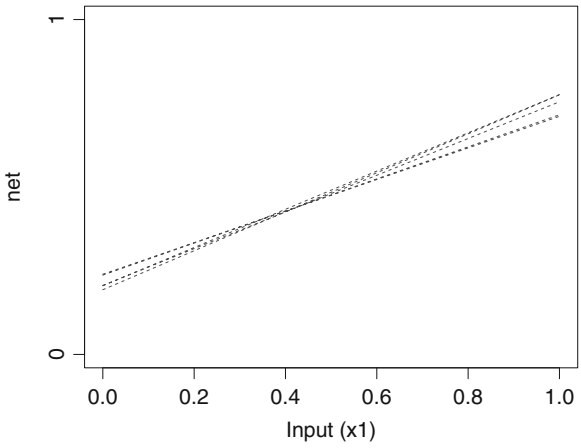


Fig. 1.37 Results provided by function net under five different training stages with parameter $\epsilon = 0.5$ (Eq. (1.10))

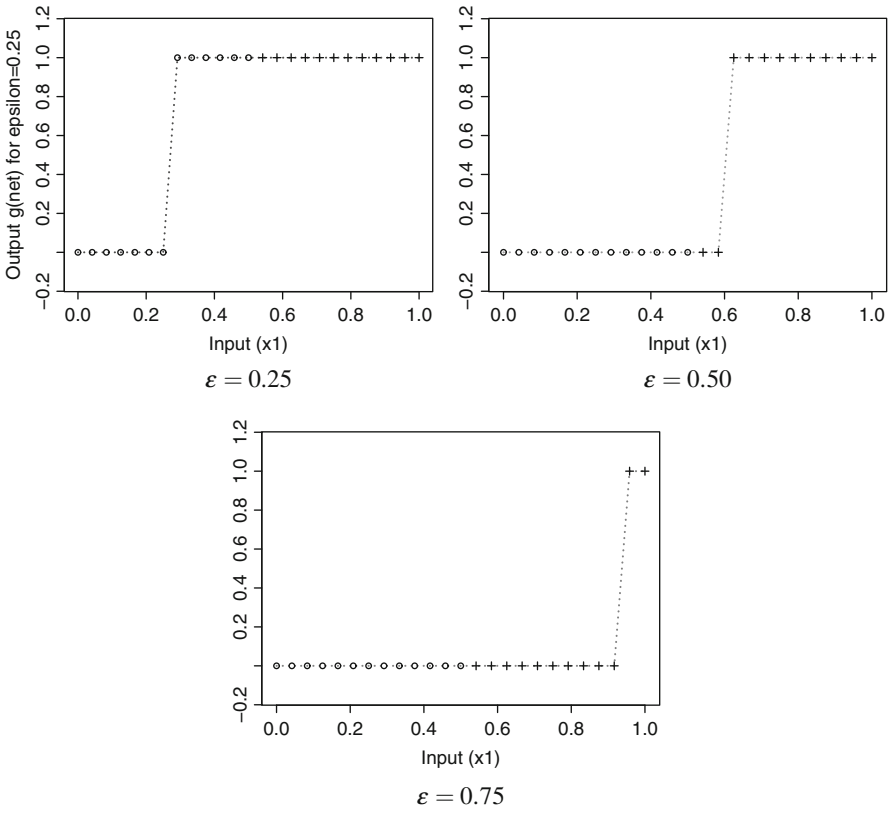


Fig. 1.38 Result provided by function net and the use of different values for parameter epsilon (Eq. (1.10)): $\epsilon = \{0.25, 0.50, 0.75\}$

That is why the squared-error function (shown in Fig. 1.40) is more adequate for the Gradient Descent method. Supervised learning algorithms assume there is a single minimum for this squared-error function. However, there are scenarios in which several local minima exist, leading to a partial and insufficient solution. For example, problem `XOR` requires two hyperplanes, as shown in Fig. 1.23. Consequently, the Perceptron would find a local solution, confirming its bias is insufficient to learn the problem `XOR`.

Listing 1.7 Functions to study how the Perceptron error behaves

```
source("perceptron.r")

perceptron.simple.error <- function(range.start=-1,
                                    range.end=1, mu=1e-10) {

   # Defining the table with examples
   # Negatives
   table = cbind(seq(0, 0.5, length=100), rep(0,100))
   # Positives
   table = rbind(table, cbind(seq(0.5+mu, 1, length=100),
                              rep(1,100)))

   # We will now define the same range for the free
       variables
   # weight and theta. Range contains 100 discretized values
   range_for_free_variables =
             seq(range.start, range.end, length=100)
   weight = range_for_free_variables
   theta = range_for_free_variables

   # Sum of errors while varying weight and theta
   error_function = matrix(0, nrow=length(weight),
                           ncol=length(theta))

   # For each weight
   for (w in 1:length(weight)) {
      # For each theta
      for (t in 1:length(theta)) {
         # Compute all net values
         net = cbind(table[,1], rep(1, nrow(table))) %*%
                                c(weight[w], theta[t])
         # Defining a vector to save the Perceptron outputs
         hat_y = rep(0, length(net))
         # Producing the output classes
         for (n in 1:length(net)) {
            # g(net) was removed to improve illustration
            hat_y[n] = net[n]
         }

         # These are the expected classes
         y = table[,2]

         # Computing the error
         error = y - hat_y

         # Saving the total error in the matrix
         error_function[w, t] = sum(error)
         # This last instruction makes positive and negative
         # terms cancel each other
      }
   }
```

```
    # Plotting the error
    filled.contour(error_function)
}

perceptron.simple.squared.error <- function(range.start=-10,
                                            range.end=10,
                                            mu=1e-10) {

    # Defining the table with examples
    table = cbind(seq(0, 0.5, length=100), rep(0, 100))
    table = rbind(table, cbind(seq(0.5+mu, 1, length=100),
                               rep(1, 100)))

    # We will now define the same range for the free
        variables
    # weight and theta. Range contains 100 discretized values
    range_for_free_variables =
            seq(range.start, range.end, length=50)
    weight = range_for_free_variables
    theta = range_for_free_variables

    # Sum of squared errors while varying weight and theta
    error_function = matrix(0, nrow=length(weight),
                            ncol=length(theta))

    # For each weight
    for (w in 1:length(weight)) {
        # For each theta
        for (t in 1:length(theta)) {
            # Compute all net values
            net = cbind(table[,1], rep(1, nrow(table))) %*%
                        c(weight[w], theta[t])
            # Defining a vector to save the Perceptron outputs
            hat_y = rep(0, length(net))
            # Producing the output classes
            for (n in 1:length(net)) {
                # g(net) was removed to improve illustration
                hat_y[n] = net[n]
            }

            # These are the expected classes
            y = table[,2]

            # This is squared to avoid negative and positive
            # values to amortize each other
            squared.error = (y - hat_y)^2

            # Saving the total squared error in the matrix
            error_function[w, t] = sum(squared.error)
        }
    }

    # We apply a log on the squared error function to improve
    # illustration, otherwise we do not see the paraboloid as
```

```
    # clear as in this form.
    filled.contour(log(error_function))
}
```

The concept of convexity is what supports the use of the squared-error function to guide learning. In a simple way, any line segment connecting two function points must lie above or on the curve to make it convex, as shown in Fig. 1.41. The most important reference for the reader is that every convex function has minima. Also notice the set of points above the function (a.k.a. epigraph) must form a convex set, as illustrated in Fig. 1.42.[7] If the supervised learning algorithm considers a convex error function, then the adaptation of free variables will eventually converge to a minimum. As cases of study, this chapter covers the Perceptron and Multilayer Perceptron (MLP) algorithms.

$$\alpha f(a) + (1-\alpha) f(b) \geq f(\alpha a + (1-\alpha)b), \quad \forall \alpha \in [0,1] \tag{1.17}$$

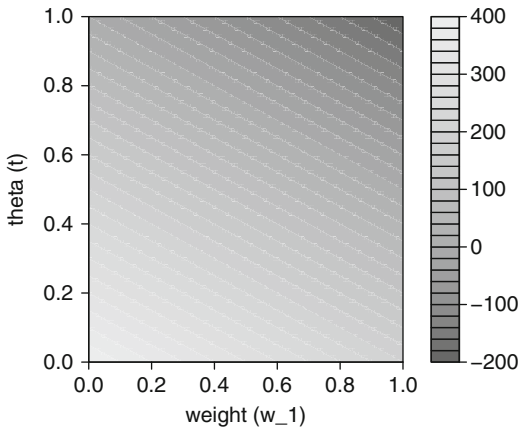


Fig. 1.39 The Perceptron: error function $y_i - \hat{y}_i$

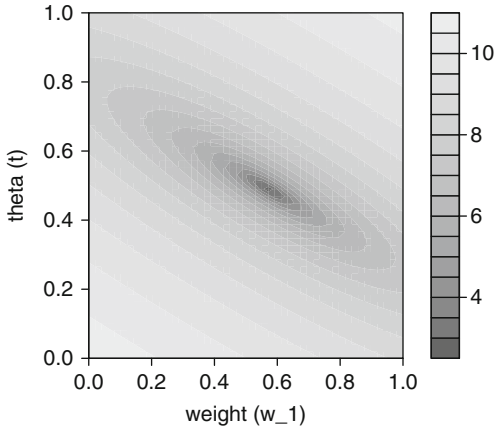


Fig. 1.40 The Perceptron: squared-error function $(y_i - \hat{y}_i)^2$

[7] The book Convex Optimization [2] is suggested.

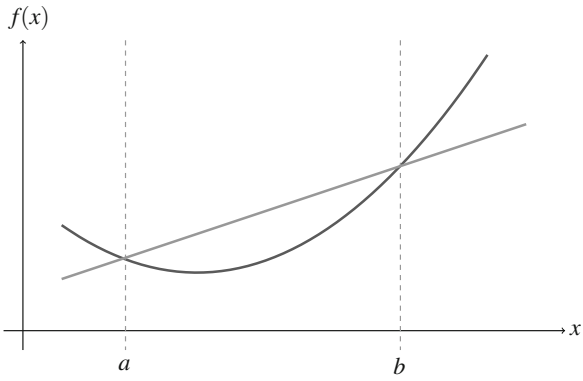


Fig. 1.41 Example of a convex function

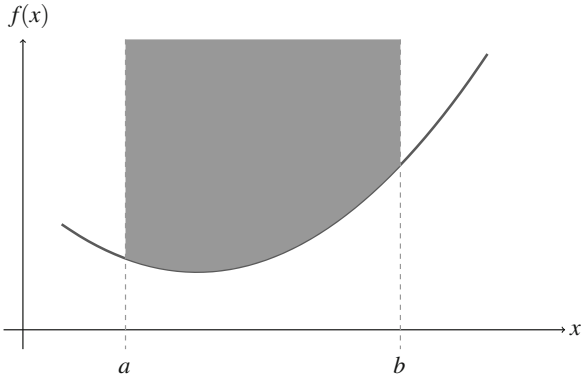


Fig. 1.42 The epigraph of a convex function must form a convex set of points

Jensen's inequality (Eq. (1.17)) formalizes when a function $f(.)$ is convex. It connects two points $f(a)$ and $f(b)$ through an affine function, so that $\alpha = 1$ corresponds to $f(a)$, and $\alpha = 0$ to $f(b)$, otherwise, provided $\alpha \in [0, 1]$, the affine point lies on the mapping connecting $f(a)$ to $f(b)$ (see Fig. 1.41).

Next, the problem `AND` is addressed for which the input and the output spaces are listed in Table 1.5, as discussed in Sect. 1.4 (see Fig. 1.20). Instead of receiving a single input variable, this problem deals with two, i.e. $x_{i,1}$ and $x_{i,2}$, in attempt to produce the expected output y_i, for every example i. Observe the Perceptron always outputs a single variable for any classification task.

In this task, the Perceptron computes the following function $\text{net} = \sum_{j=1}^{2} x_{i,j} w_j + \theta = x_{i,1} w_1 + x_{i,2} w_2 + \theta$ and then applies the activation function to produce classes $\hat{y}_i = g(\text{net}),\ \forall i$ (Eq. (1.10)). Listing 1.8 provides two additional functions to implement the problem `AND`, which also considers our previous codes. By running function `perceptron.run.AND()`, the Perceptron is trained to infer a model, whose hyperplane is plotted afterwards (Fig. 1.43). We suggest the reader to run function `perceptron.run.AND()` several times and observe how the hyperplane changes, due to the algorithm converges to different solutions given weights and theta are randomly initialized.

Listing 1.8 Additional functions to implement the problem AND

```
source("perceptron.r")

# This is an example of learning the problem AND
#
# To run this example:
#
#       1) Open the R Statistical Software
#       2) source("perceptron-AND.r")
#       3) perceptron.run.AND()
#
perceptron.run.AND <- function() {

   # This is a table with all possible examples
   # for the problem AND. In this case we will
   # use the same table for training and testing,
   # just because we know all possible binary
   # combinations
   table = matrix(c(0, 0, 0,  # 0 AND 0 = 0
                    0, 1, 0,  # 0 AND 1 = 0
                    1, 0, 0,  # 1 AND 0 = 0
                    1, 1, 1), # 1 AND 1 = 1
                  nrow=4,
                  ncol=3,
                  byrow=TRUE)

   # Training the Perceptron to find weights and theta
   training.result = perceptron.train(table)

   # Testing the Perceptron with the weights and theta found
   perceptron.test(table, training.result$weights,
                          training.result$theta)

   # Plotting the hyperplane found
   perceptron.hyperplane.plot(training.result$weights,
                              training.result$theta)
}

# This function plots the hyperplane found for a given
# classification task with two input variables only.
# Variables range.start and range.end define the interval
# for variables composing the problem. The problem AND
# has two variables composing each example i, which are
# x_i,1 and x_i,2
perceptron.hyperplane.plot <- function(weights, theta,
                                       range.start=0,
                                       range.end=1) {

   # Variable weights define the number of input variables
   # we have, so we can use this information to create
   # axes in a multidimensional input space in order to
   # see how inputs modify the output class provided by
   # the Perceptron
```

```
    nVars = length(weights)

    # We will now define the same range for every input
    # variable. This range will contain 100 discretized
    # values
    range_of_every_input_variable =
                  seq(range.start, range.end, length=100)

    x_1 = range_of_every_input_variable
    x_2 = range_of_every_input_variable

    # Function outer combines every possible value for
    # variable x_1 against every possible value for x_2.
    # Observe they are continuous values which were never
    # seen (we expect either 0 or 1) by this Perceptron
    # during the training stage. Also observe value 1
    # inside the cbind, which refers to the 1 * theta while
    # computing function net. Operation %*% corresponds
    # to the dot product.
    all_nets = outer(x_1,x_2, function(x,y) {
                      cbind(x,y,1) %*% c(weights,theta) } )

    # This variable all_nets contains all net values for
    # every combination between variables x_1 and x_2.
    # Variable y will contain the Perceptron outputs after
    # applying the heaviside function
    y = matrix(0, nrow=nrow(all_nets), ncol=ncol(all_nets))
    for (row in 1:nrow(all_nets)) {
        for (col in 1:ncol(all_nets)) {
            y[row, col] = g(all_nets[row, col])
        }
    }

    # Plotting the hyperplane found by the Perceptron
    filled.contour(x_1, x_2, y)
}
```

The reader should also apply the Perceptron on the problem XOR to notice that it never finds a suitable hyperplane. It will fail for at least 25% of examples because when the hyperplane answers correctly for one of the classes, it will misclassify half of examples associated to the other class, as seen in Fig. 1.44. Since the error does not converge to zero, the threshold must be significantly increased so that the hyperplane can be plotted. As already discussed, the hyperplane will change along every run, providing different classification answers.

Problems such as XOR motivated the design of supervised learning algorithms using multiple hyperplanes. The Multilayer Perceptron (MLP) is a remarkable example among all other algorithms, what motivated its discussion along the next section.

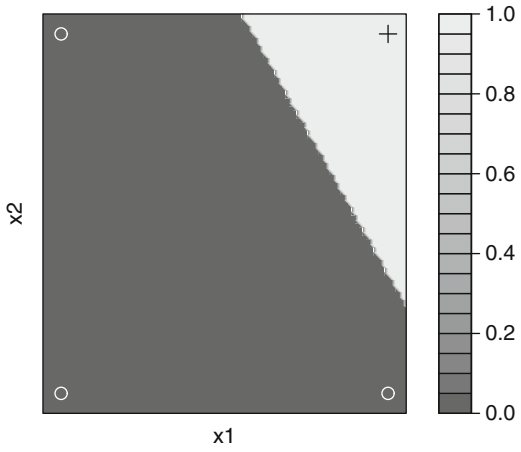


Fig. 1.43 Hyperplane found by the Perceptron trained on the problem AND

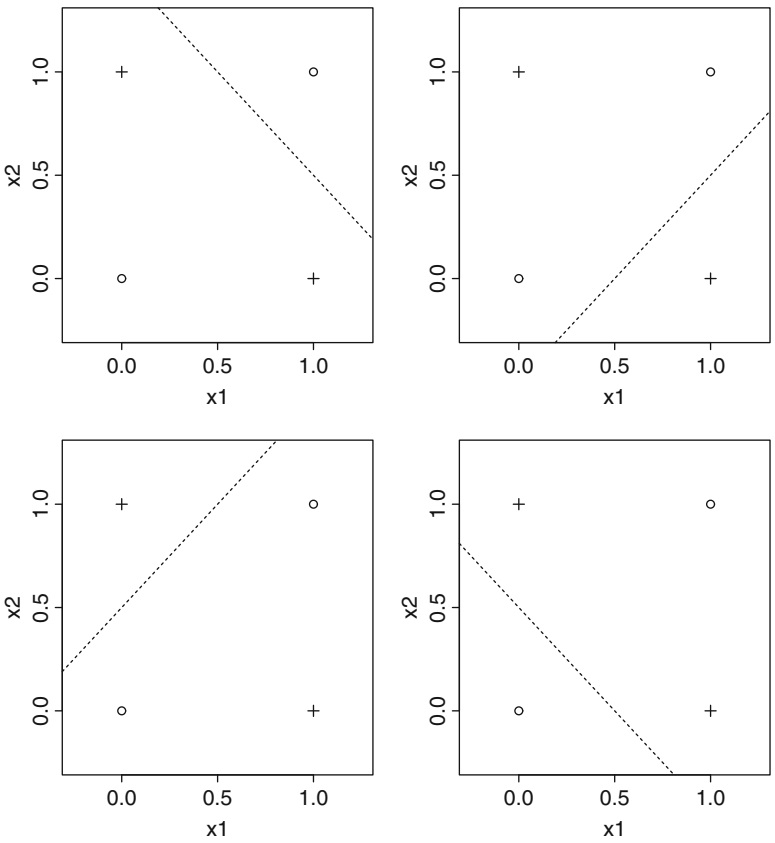


Fig. 1.44 Different attempts to use a single hyperplane to solve the problem XOR

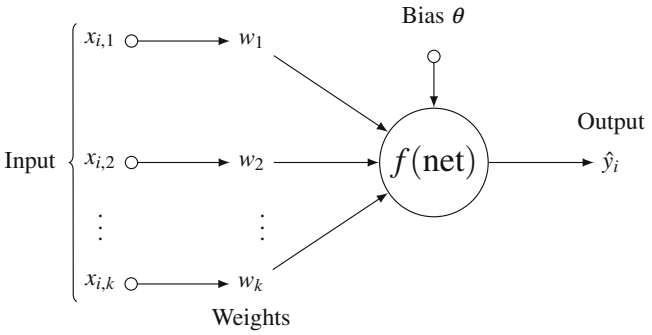


Fig. 1.45 A single unit or neuron of the Multilayer Perceptron, in which $f(.)$ is the activation function

1.5.2 Multilayer Perceptron

The problem XOR is a toy example to motivate solutions for more complex classification tasks. Those complex scenarios require supervised learning algorithms to employ multiple hyperplanes, such as the Multilayer Perceptron (MLP). MLP considers multiple units (or neurons) of the Perceptron algorithm organized in consecutive layers.

An MLP unit (or neuron) is illustrated in Fig. 1.45, showing input variables $x_{i,1}, \ldots, x_{i,k}$ as well as the output variable $\hat{y}_i$ for some example i, weights $w_1, \ldots, w_k$ and θ. Similarly to the Perceptron, an MLP neuron computes function $\text{net} = \sum_{j=1}^{k} x_{i,j} w_j + \theta$, however its output is given by an arbitrary activation function $f(\text{net})$. The most commonly used activation function is the sigmoid, whose effect is illustrated in conjunction with the heaviside function in Fig. 1.46. Notice such activation provides continuous outputs, in contrast with discrete ones from the heaviside function.

$$f(\text{net}) = \frac{1}{1 + e^{-\text{net}}} \tag{1.18}$$

Equation (1.18) is differentiable, making possible the use of the Gradient Descent (GD) method to address the MLP learning. In contrast, the heaviside function $g(\text{net})$ had to be disconsidered while formulating the Perceptron (or it might be differentiated in parts).

Units or neurons are connected to form an architecture such as the one illustrated in Fig. 1.47, which contains two neurons in the **input layer**, two others in the **hidden layer**, and a single one in the **output layer**. The number of neurons at the input layer must always match the number of input variables. For example, for the problem XOR, two neurons are needed in the input layer. Neurons in this first layer build identity functions, i.e., their output is the same value they receive as input, so weights and theta are not necessary.

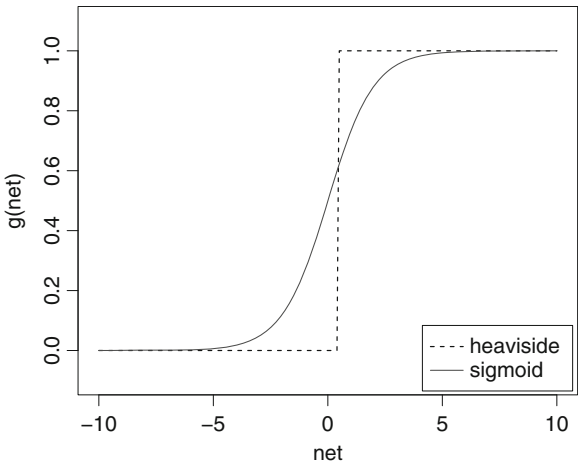


Fig. 1.46 The outputs produced by the heaviside versus the sigmoid function according to the input value net

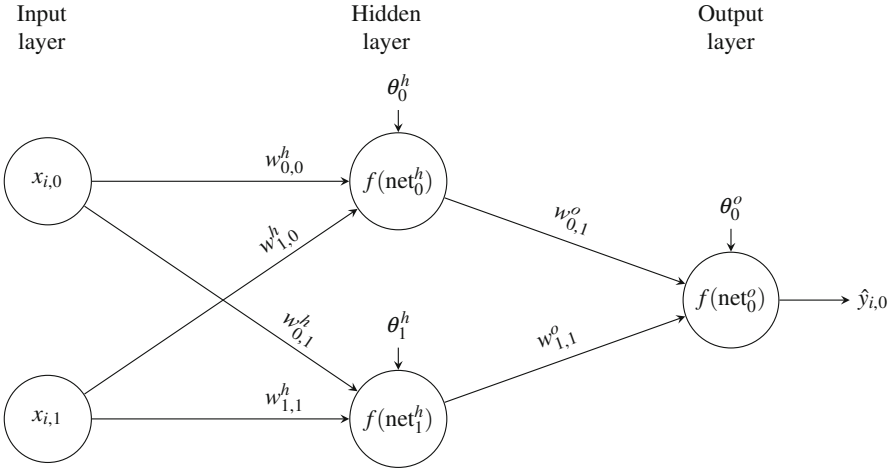


Fig. 1.47 An example of an MLP architecture with two neurons at the input layer, two others at the hidden layer, and, finally, a single neuron at the output layer

The hidden layer defines the number of hyperplanes to shatter (or divide) the input space. Again, the problem XOR requires two hyperplanes, consequently this layer must be set with two neurons. There are several problems that cannot be plotted due to the input space dimensionality, thus classification results must be considered in order to set an adequate number of neurons. At last, the output layer must contain sufficient neurons to encode all output classes. As XOR is a two-class problem, a single output neuron is enough, finally concluding the MLP architecture to tackle this specific classification task.

For such architecture, the following free variables are set:

1. Input layer—there is no weight nor theta to adapt, because they act as identity functions. They output the same value received as input;
2. Hidden layer—every neuron is identified by a number, in this situation neurons are indexed by $j = \{0, 1\}$, so we have weights $w^h_{j,0}$ and $w^h_{j,1}$, and θ^h_j, where superscript h indicates variables belonging to the hidden layer, and the second subscript (either 0 or 1) indexes the source neuron of the input layer;
3. Output layer—every neuron is also identified by a number, however, in this case, there is a single neuron $k = \{0\}$. Weights for neuron k are $w^o_{k,0}$ and $w^o_{k,1}$, in which superscript o means the output layer, and the second subscript (either 0 or 1) is associated to the source neuron of the hidden layer.

Learning is therefore the process of adapting all those free variables in attempt to answer the correct output classes for the problem XOR. As piece of information, more than a single hidden layer may be used to shatter the input space, however this book will not cover it.

How should those free variables be adapted? Again, a loss function such as squared-error $(y_i - \hat{y}_i)^2$ may be computed along the training set. For the Perceptron, this is a convex function due to the simplification $\hat{y}_i = \text{net}_i$, which is easily seen once net_i is linear and the expected class y_i is a constant, therefore the result is indeed a power of two for some linear function, forming a convex function.

By using a differentiable activation function, such as in MLP, no simplification is required. For the usual MLP, the squared-error loss function takes a constant y_i minus the sigmoid function $f(\text{net}_i)$, producing a quasi-convex function which also allows learning but under some conditions. Figure 1.48 illustrates quasi-convex[8] functions typically associated with the Multilayer Perceptron. Jensen's Inequality 1.17 connects a pair of points $f(a)$ to $f(b)$ so the region between them can be evaluated in terms of convexity. For example, Fig. 1.49 shows the convex interval as a shaded area and the non-convex intervals are outside.

The reader should plot function $(c - f(\text{net}))^2$ for values of net $\in \mathbb{R}$ varying constant c.[9] Observe how net influences in the squared-error function, as depicted in Fig. 1.48. If net is too great or too small, observe we may be out of the convex region and any neighborhood around provides no direction for the GD method, i.e., the derivative of this error function is numerically equal to zero.

Figure 1.49 shows a more interesting situation, in which besides interval $[a, b]$ is not convex, the GD method can be applied once the derivative is not null. This scenario can indeed happen to MLP and it is not difficult to understand why. Consider one has two variables $x_{i,1}$ and $x_{i,2}$ for some input example i. Let $x_{i,1}$ be in range $[10^5, 10^6]$ and $x_{i,2}$ be in $[-1, 1]$, and weights and thetas be randomly initialized in interval $[-1, 1]$. Consequently, function net $= x_{i,1}w_1 + x_{i,2}w_2 + \theta$ will produce very large or very small values, impacting the convergence to the minimal squared error. To solve this issue, either w_1 could be initialized in a smaller interval

[8] We suggest the book Convex Optimization [2] for further references and to complement studies.

[9] Constant c is the expected class and it is in range [0, 1].

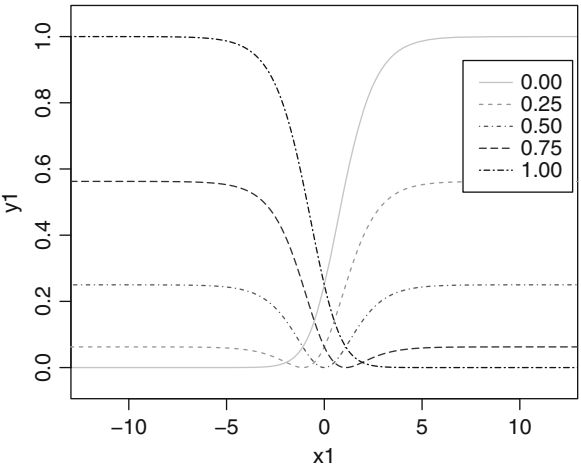


Fig. 1.48 Illustration of quasi-convex functions produced when using the squared-error loss function for the Multilayer Perceptron using different values for $c = \{0, 0.25, 0.5, 0.75, 1\}$ in a function $c - f(\text{net})$

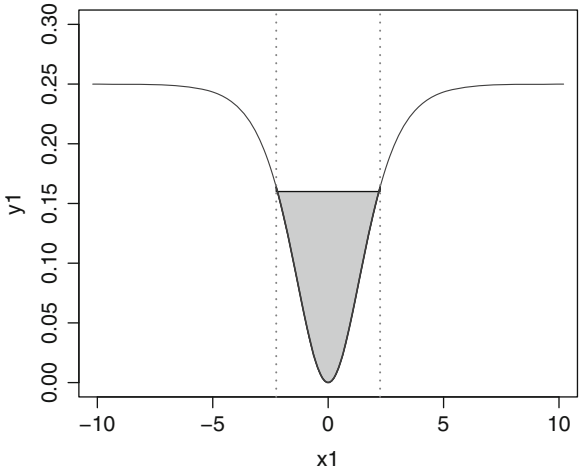


Fig. 1.49 The effect of net in the quasi-convex function produced by using the squared-error loss function for the Multilayer Perceptron, given constant $c = 0.5$. The region below the line is quasi-convex in terms of net

to force $x_{i,1}$ to fit in the same scale as $x_{i,2}w_2$ or all inputs could be normalized in range $[-1, 1]$, so one input will not dominate net. That is why normalization is so used while tackling classification problems with neural networks. By ensuring that, we can proceed with the GD method to find the squared-error minimum.

We formalize MLP with the squared-error function:

$$E_i^2 = (y_i - f(\text{net}_i))^2 , \tag{1.19}$$

in which $f(\text{net}_i)$ corresponds to the output MLP produces for example i. Rewriting term $f(\text{net}_i)$:

$$E_i^2 = \sum_k \left(y_{i,k} - f_k^o\left(\text{net}_{i,k}^o\right)\right)^2 = \left(y_{i,k} - f_k^o\left[\sum_j f_j^h(\text{net}_j^h) w_{k,j}^o + \theta_k^o\right]\right)^2, \tag{1.20}$$

in which $f_k^o(\text{net}_{i,k}^o)$ is the sigmoid function (Eq. (1.18)) applied by the kth MLP neuron at the output layer, that is why the superscript o is used (meaning output); $f_j^h(\text{net}_j^h)$ refers to the output produced by neuron j at the hidden layer (superscript h); $w_{k,j}^o$ is the weight from neuron k at the output layer receiving input $f_j^h(\text{net}_j^h)$ from the hidden layer (first subscript k indexes the owner neuron, and subscript j identifies the neuron from the preceding layer which provides the input); θ_k^o is the bias term from neuron k at the output layer; the internal summation indexed with j is responsible for computing $\text{net}_{i,k}^o$ (Eq. (1.19) defined this same term as net_i for simplification purposes), thus:

$$\text{net}_{i,k}^o = \sum_j f_j^h(\text{net}_j^h) w_{k,j}^o + \theta_k^o,$$

and, finally, the external summation in terms of k computes the overall divergences for all output neurons. In particular, for the problem XOR, term $k = 0$ meaning there is only one output neuron. The same happens with the expected output $y_{i,k}$, which is a single value for every example i, that is why we used the simplified version y_i in Eq. (1.19).

This squared-error function (Eq. (1.20)) allows us to proceed with the GD method to train all MLP free variables. For XOR, the GD method will consider:

1. Equation (1.21) to adapt every weight $w_{j,l}^h$ connecting input neuron l to hidden neuron j;
2. Equation (1.22) to adapt every weight $w_{k,j}^o$ connecting hidden neuron j to output neuron k.

$$w_{j,l}^h(t+1) = w_{j,l}^h(t) - \eta \frac{\partial E_i^2}{\partial w_{j,l}^h} \tag{1.21}$$

$$w_{k,j}^o(t+1) = w_{k,j}^o(t) - \eta \frac{\partial E_i^2}{\partial w_{k,j}^o} \tag{1.22}$$

To complement, Eqs. (1.23) and (1.24) provide the GD method to adapt the free variable θ for every neuron at either the hidden or the output layer along iterations:

$$\theta_j^h(t+1) = \theta_j^h(t) - \eta \frac{\partial E_i^2}{\partial \theta_j^h}, \tag{1.23}$$

$$\theta_k^o(t+1) = \theta_k^o(t) - \eta \frac{\partial E_i^2}{\partial \theta_k^o}. \tag{1.24}$$

Despite the Gradient Descent equations are the same for weights in both layers as well as for thetas, the partial derivatives:

$$\frac{\partial E_i^2}{\partial w_{j,l}^h}, \frac{\partial E_i^2}{\partial w_{k,j}^o}, \frac{\partial E_i^2}{\partial \theta_j^h}, \text{ and } \frac{\partial E_i^2}{\partial \theta_k^o}$$

change. Consequently, we must compute the partial derivatives for this problem `XOR` to exemplify the MLP formulation, however this solution still supports a general-purpose algorithm.

The partial derivatives for the output layer are found in advance, once they are simpler to obtain:

$$\begin{aligned}\frac{\partial E_i^2}{\partial w_{k,j}^o} &= \frac{\partial \sum_k (y_{i,k} - \hat{y}_{i,k})^2}{\partial w_{k,j}^o} \\ &= \frac{\sum_k \left(y_{i,k} - f_k^o\left(\sum_j f_j^h(\text{net}_j^h) w_{k,j}^o + \theta_k^o\right)\right)^2}{\partial w_{k,j}^o},\end{aligned} \tag{1.25}$$

given a training example i, neuron k at the output layer, and neuron j at the hidden layer.

The differentiation employs the chain rule as follows:

$$\frac{\partial E_i^2}{\partial w_{k,j}^o} = \sum_k 2\left(y_{i,k} - \hat{y}_{i,k}\right) \frac{\partial \left(y_{i,k} - \hat{y}_{i,k}\right)}{\partial w_{k,j}^o},$$

as $y_{i,k}$ is a constant defining the expected class to be produced by output neuron k, assuming zero while deriving in terms of $w_{k,j}^o$:

$$\frac{\partial E_i^2}{\partial w_{k,j}^o} = \sum_k 2(y_{i,k} - \hat{y}_{i,k}) \frac{\partial - \hat{y}_{i,k}}{\partial w_{k,j}^o},$$

thus, we still need to solve the derivative $\frac{\partial - \hat{y}_{i,k}}{\partial w_{k,j}^o}$. As the reader may recall, $\hat{y}_{i,k} = f_k^o(\text{net}_{i,k}^o)$, i.e., the output value produced by output neuron k is the result of the sigmoid function, defined in Eq. (1.18):

$$\frac{\partial - \hat{y}_{i,k}}{\partial w_{k,j}^o} = \frac{\partial - f_k^o(\text{net}_{i,k}^o)}{\partial w_{k,j}^o}$$

$$= \frac{\partial - (1 + e^{-\text{net}_{i,k}^o})^{-1}}{\partial w_{k,j}^o}$$

$$= \frac{\partial - \left(1 + e^{-\left[\sum_j f_j^h(\text{net}_j^h) w_{k,j}^o + \theta_k^o\right]}\right)^{-1}}{\partial w_{k,j}^o}.$$

We know the sigmoid function (Eq. (1.18)) has the following derivative in terms of net:

$$\frac{\partial f(\text{net})}{\partial \text{net}} = f(\text{net})(1 - f(\text{net})),$$

what simplifies our formulation:

$$\frac{\partial - f_k^o(\text{net}_{i,k}^o)}{\partial w_{k,j}^o} = -\left[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \frac{\partial \text{net}_{i,k}^o}{\partial w_{k,j}^o} \right],$$

so, we still need to find the following partial derivative:

$$\frac{\partial \text{net}_{i,k}^o}{\partial w_{k,j}^o} = \frac{\partial \sum_j f_j^h(\text{net}_{i,j}^h) w_{k,j}^o + \theta_k^o}{\partial w_{k,j}^o} = f_j^h(\text{net}_{i,j}^h).$$

Connecting all terms, the update rule for weights at the output layer is:

$$w_{k,j}^o(t+1) = w_{k,j}^o(t) - \eta \frac{\partial E_i^2}{\partial w_{k,j}^o}$$

$$w_{k,j}^o(t+1) = w_{k,j}^o(t) - \eta\, 2(y_i - \hat{y}_{i,k}) \frac{\partial - \left[f_k^o(\text{net}_{i,k}^o) \right]}{\partial w_{k,j}^o}$$

$$w_{k,j}^o(t+1) = w_{k,j}^o(t) - \eta\, 2(y_i - f_k^o(\text{net}_{i,k}^o)) \left(- \left[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \right] \right) \frac{\partial \text{net}_{i,k}^o}{\partial w_{k,j}^o}$$

$$w_{k,j}^o(t+1) = w_{k,j}^o(t) - \eta\, 2(y_i - f_k^o(\text{net}_{i,k}^o)) \left(- \left[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \right] \right) f_j^h(\text{net}_{i,j}^h).$$

Similarly, the Gradient Descent formulated for theta from output neuron k is:

$$\theta_k^o(t+1) = \theta_k^o(t) - \eta\ 2(y_i - f_k^o(\text{net}_{i,k}^o))\left(-\left[f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o))\right]\right)1,$$

which has another term instead of $f_j^h(\text{net}_{i,j}^h)$, once the derivative:

$$\frac{\partial \text{net}_{i,k}^o}{\partial \theta_k^o} = \frac{\partial\left[\sum_j f_j^h(\text{net}_{i,j}^h) w_{k,j}^o + \theta_k^o\right]}{\partial \theta_k^o} = 1,$$

results in the number 1. Finally, we have the update rules for weights and thetas at the output layer.

Next, the corresponding rules for the hidden layer must be found. First we detail the derivative of the squared-error in terms of weights $w_{j,l}^h$:

$$\begin{aligned}
\frac{\partial E_i^2}{\partial w_{j,l}^h} &= \frac{\partial \sum_k (y_{i,k} - \hat{y}_{i,k})^2}{\partial w_{j,l}^h} \\
&= \frac{\sum_k \left(y_{i,k} - f_k^o\left(\sum_j f_j^h(\text{net}_{i,j}^h) w_{k,j}^o + \theta_k^o\right)\right)^2}{\partial w_{j,l}^h} \\
&= \frac{\sum_k \left(y_{i,k} - f_k^o\left(\sum_j f_j^h\left(\sum_l x_{i,l} w_{j,l}^h + \theta_j^h\right) w_{k,j}^o + \theta_k^o\right)\right)^2}{\partial w_{j,l}^h},
\end{aligned}$$

given:

$$\text{net}_{i,j}^h = \sum_l x_{i,l} w_{j,l}^h + \theta_j^h. \tag{1.26}$$

Thus, the partial derivative is defined as:

$$\begin{aligned}
\frac{\partial E_i^2}{\partial w_{j,l}^h} &= 2 \sum_k (y_{i,k} - \hat{y}_{i,k}) \left(\frac{\partial y_{i,k} - \hat{y}_{i,k}}{\partial w_{j,l}^h}\right) \\
&= 2 \sum_k (y_{i,k} - f_k^o(\text{net}_{i,k}^o)) \left(\frac{\partial - f_k^o(\text{net}_{i,k}^o)}{\partial w_{j,l}^h}\right),
\end{aligned}$$

having:

$$\frac{\partial - f_k^o(\text{net}_{i,k}^o)}{\partial w_{j,l}^h} = -\frac{\partial f_k^o(\text{net}_{i,k}^o)}{\partial \text{net}_{i,k}^o} \frac{\partial \text{net}_{i,k}^o}{\partial w_{j,l}^h} = -f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \frac{\partial \text{net}_{i,k}^o}{\partial w_{j,l}^h},$$

in which:

$$\frac{\partial f_k^o(\text{net}_{i,k}^o)}{\partial \text{net}_{i,k}^o} = f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)),$$

for the sigmoid function.

Now we solve:

$$\frac{\partial \text{net}_{i,k}^o}{\partial w_{j,l}^h},$$

which is formulated in terms of the chain rule:

$$\begin{aligned}\frac{\partial \text{net}_{i,k}^o}{\partial w_{j,l}^h} &= \frac{\partial \left[\sum_j f_j^h(\sum_l x_{i,l} w_{j,l}^h + \theta_j^h) w_{k,j}^o + \theta_k^o\right]}{\partial w_{j,l}^h} \\ &= \frac{\partial f_j^h(\text{net}_{i,j}^h)}{\partial \text{net}_{i,j}^h} \frac{\partial \text{net}_{i,j}^h}{\partial w_{j,l}^h},\end{aligned}$$

having $\text{net}_{i,j}^h$ defined in Eq. (1.26), obtaining:

$$\begin{aligned}\frac{\partial - f_k^o(\text{net}_{i,k}^o)}{\partial w_{j,l}^h} &= -\frac{\partial f_k^o(\text{net}_{i,k}^o)}{\partial \text{net}_{i,k}^o} \frac{\partial \text{net}_{i,k}^o}{\partial w_{j,l}^h} \\ &= -f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \frac{\partial \text{net}_{i,k}^o}{\partial w_{j,l}^h} \\ &= -f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \frac{\partial f_j^h(\text{net}_{i,j}^h)}{\partial \text{net}_{i,j}^h} \frac{\partial \text{net}_{i,j}^h}{\partial w_{j,l}^h} \\ &= -f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \left[f_j^h(\text{net}_{i,j}^h)(1 - f_j^h(\text{net}_{i,j}^h))\right] \frac{\partial \text{net}_{i,j}^h}{\partial w_{j,l}^h} \\ &= -f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \left[f_j^h(\text{net}_{i,j}^h)(1 - f_j^h(\text{net}_{i,j}^h))\right] x_{i,l},\end{aligned}$$

and, finally:

$$\begin{aligned}\frac{\partial E_i^2}{\partial w_{j,l}^h} &= 2 \sum_k \left(y_{i,k} - f_k^o(\text{net}_{i,k}^o)\right) \left(\frac{\partial - f_k^o(\text{net}_{i,k}^o)}{\partial w_{j,l}^h}\right), \\ &= 2 \sum_k (y_{i,k} - f_k^o(\text{net}_{i,k}^o)) \\ &\quad \left(-f_k^o(\text{net}_{i,k}^o)(1 - f_k^o(\text{net}_{i,k}^o)) \left[f_j^h(\text{net}_{i,j}^h)(1 - f_j^h(\text{net}_{i,j}^h))\right] x_{i,l}\right).\end{aligned}$$

Connecting all terms, the update rule for weights at the hidden layer is:

$$w^h_{j,l}(t+1) = w^h_{j,l}(t) - \eta \frac{\partial E_i^2}{\partial w^h_{j,l}}$$

$$w^h_{j,l}(t+1) = w^h_{j,l}(t) - \eta\, 2 \sum_k (y_i - \hat{y}_{i,k}) \frac{\partial - \hat{y}_{i,k}}{\partial w^h_{j,l}}$$

$$w^h_{j,l}(t+1) = w^h_{j,l}(t) - \eta\, 2 \sum_k (y_i - f^o_k(\text{net}^o_{i,k})) \frac{\partial - f^o_k(\text{net}^o_{i,k})}{\partial w^h_{j,l}}$$

$$\begin{aligned} w^h_{j,l}(t+1) = w^h_{j,l}(t) & \\ & - \eta\, 2 \sum_k (y_i - f^o_k(\text{net}^o_{i,k})) \\ & \left(-f^o_k(\text{net}^o_{i,k})(1 - f^o_k(\text{net}^o_{i,k})) \left[f^h_j(\text{net}^h_{i,j})(1 - f^h_j(\text{net}^h_{i,j})) \right] x_{i,l} \right), \end{aligned}$$

and, because some terms are independent of index k, we can simplify the previous formulation as follows:

$$\begin{aligned} w^h_{j,l}(t+1) = w^h_{j,l}(t) & \\ & - \eta\, 2 \sum_k (y_i - f^o_k(\text{net}^o_{i,k})) \\ & (-f^o_k(\text{net}^o_{i,k})(1 - f^o_k(\text{net}^o_{i,k})) \left[f^h_j(\text{net}^h_{i,j})(1 - f^h_j(\text{net}^h_{i,j})) \right] x_{i,l}) \end{aligned}$$

$$\begin{aligned} w^h_{j,l}(t+1) = w^h_{j,l}(t) & \\ & - \eta\, 2 \left[f^h_j(\text{net}^h_{i,j})(1 - f^h_j(\text{net}^h_{i,j})) \right] \\ & x_{i,l} \sum_k (y_i - f^o_k(\text{net}^o_{i,k}))(-f^o_k(\text{net}^o_{i,k})(1 - f^o_k(\text{net}^o_{i,k}))). \end{aligned}$$

Similarly, the Gradient Descent method formulated for theta of hidden neuron j is:

$$\begin{aligned} \theta^h_j(t+1) = \theta^h_j(t) & \\ & - \eta\, 2 \left[f^h_j(\text{net}^h_{i,j})(1 - f^h_j(\text{net}^h_{i,j})) \right] \\ & (1) \sum_k (y_i - f^o_k(\text{net}^o_{i,k}))(-f^o_k(\text{net}^o_{i,k})(1 - f^o_k(\text{net}^o_{i,k}))), \end{aligned}$$

given:

$$\frac{\partial \text{net}^h_{i,j}}{\partial \theta^h_j} = 1.$$

We omit constant 2 in all gradient rules, because the gradient step η is already a constant set up by the user. After having all the Gradient Descent rules, the MLP source code, using the R language, is detailed in Listing 1.9.

Listing 1.9 The Multilayer Perceptron implementation

```
# This is the MLP sigmoid activation function
f <- function(net) {
   ret = 1.0 / (1.0 + exp(-net))
   return (ret)
}

# This function is used to build up the MLP architecture, i.
    e.,
# the neurons contained in the hidden and the output layers
# with their respective weights and thetas randomly
    initialized.
mlp.architecture <- function(input.layer.size = 2,
      hidden.layer.size = 2,
      output.layer.size = 1,
      f.net = f) {

   # Here we create a list to contain the layers information
   layers = list()

   # This is the hidden layer in which weights and thetas
   # were initialized in a random manner (using runif) in
   # interval [-1,1]. Term input.layer.size+1 refers to
   # the number of neurons in the input layer (a weight
   # per unit), plus an additional element to define theta
   layers$hidden = matrix(runif(min=-1, max=1,
                n=hidden.layer.size*(input.layer.size+1)),
                nrow=hidden.layer.size,
                ncol=input.layer.size+1)

   # The same as the hidden layer happens here, but for the
   # output layer
   layers$output = matrix(runif(min=-1, max=1,
               n=output.layer.size*(hidden.layer.size+1)),
                nrow=output.layer.size,
                ncol=hidden.layer.size+1)

   # Defining a list to return everything:
   # - the number of units or neurons at the input layer
   # - the number of units at the hidden layer
   # - the number of units at the output layer
   # - layers information (including weights and thetas)
   # - the activation function used is also returned
   ret = list()
   ret$input.layer.size = input.layer.size
   ret$hidden.layer.size = hidden.layer.size
   ret$output.layer.size = output.layer.size
   ret$layers = layers
```

```
    ret$f.net = f.net

    return (ret)
}

# This function produces the MLP output after providing
    input
# values. Term architecture refers to the model produced by
# function mlp.architecture. Term dataset corresponds to the
# examples used as input to the MLP. Term p is associated to
# the identifier of the current example being forwarded.
forward <- function(architecture, dataset, p) {

    # Organizing dataset as input examples x
    x = matrix(dataset[,1:architecture$input.layer.size],
           ncol=architecture$input.layer.size)
    # Organizing dataset as expected classes y associated to
    # input examples x
    y = matrix(
      dataset[,(architecture$input.layer.size+1):ncol(dataset
          )],
      nrow=nrow(x))

    # Submitting the p-th input example to the hidden layer
    net_h = architecture$layers$hidden %*%
                    c(as.vector(ts(x[p,])), 1)
    f_net_h = architecture$f.net(net_h)

    # Hidden layer outputs as inputs for the output layer
    net_o = architecture$layers$output %*% c(f_net_h, 1)
    f_net_o = architecture$f.net(net_o)

    # Here we have the final results produced by the MLP
    ret = list()
    ret$f_net_h = f_net_h
    ret$f_net_o = f_net_o

    return (ret)
}

# This function is responsible for training, i.e., adapting
# weights and thetas for every neuron (or unit). It
    basically
# applies the Gradient Descent Method.
backpropagation <- function(architecture, dataset,
                            eta=0.1, threshold=1e-3) {

    x = matrix(dataset[,1:architecture$input.layer.size],
           ncol=architecture$input.layer.size)
    y = matrix(
      dataset[,(architecture$input.layer.size+1):ncol(dataset
          )],
      nrow=nrow(x))
```

```
cat("Input data...\n")
print(x)

cat("Expected output...\n")
print(y)

cat("Enter to start running...")
readline()

squared_error = threshold * 2

# This loop will run until the average squared error is
# below some threshold value.
while (squared_error > threshold) {

   # Initializing the squared error to measure the loss
   # for all examples in the training set
   squared_error = 0

   # For every example at index (row) p
   for (p in 1:nrow(x)) {

      # Applying the input example at index p
      f = forward(architecture, dataset, p)

      # Getting results to adapt weights and thetas
      error = (y[p,] - f$f_net_o)

      # Computing term delta for the output layer
      # which simplifies next computations involved
      # in the Gradient Descent method
      delta_o = error * f$f_net_o * (1-f$f_net_o)

      # This is the squared error used as stop criterion.
      # Term sum(error^2) is used because the last layer
      # (i.e., the output layer) may have more than a
      # single neuron. We also use a power of two to
      # ensure negative and positive values do not
      # nullify each other.
      squared_error = squared_error + sum(error^2)

      # Computing term delta for the hidden layer
      w_o = architecture$layers$output[,
               1:architecture$hidden.layer.size]
      delta_h = (f$f_net_h * (1 - f$f_net_h)) *
                sum(as.vector(delta_o) * as.vector(w_o))

      # Adapting weights and thetas at the output layer
      architecture$layers$output =
         architecture$layers$output + eta * delta_o %*%
            c(f$f_net_h, 1)

      # Adapting weights and thetas at the hidden layer
      architecture$layers$hidden =
```

```
            architecture$layers$hidden + eta * delta_h %*%
              c(x[p,], 1)
      }

      # Dividing the total squared error by nrow to find
      # the average which we decided to use as stop
          criterion
      squared_error = squared_error / nrow(x)

      # Printing the average squared error out
      cat("Squared error = ", squared_error, "\n")
   }

   # Returning the trained architecture, which can now
   # be used for execution.
   return (architecture)
}

# This function is used to test the MLP
mlp.test <- function(architecture, dataset, debug=T) {

   # Organizing dataset as input examples x
   x = matrix(dataset[,1:architecture$input.layer.size],
              ncol=architecture$input.layer.size)

   # Organizing dataset as expected classes y associated to
   # input examples x
   y = matrix(
     dataset[,(architecture$input.layer.size+1):ncol(dataset
         )],
     nrow=nrow(x))

   cat("Enter to start testing...")
   readline()

   output = NULL

   # For every example at index (row) p
   for (p in 1:nrow(x)) {

      # Applying the input example at index p
      f = forward(architecture, dataset, p)

      # If debug is true, show all information
      # regarding classification
      if (debug) {
         cat("Input pattern =", as.vector(x[p,]),
             " Expected =", as.vector(y[p,]),
            " Predicted =", as.vector(f$f_net_o), "\n")
      }

      # Concatenating all output values as rows in a matrix,
      # so we can check them out later.
      output = rbind(output, as.vector(f$f_net_o))
```

```
    }

    # Returning results
    return (output)
}

# This function is useful to produce a discrete
# (either yes or no) hyperplane to shatter the
# input space of examples
discretize.hiperplane <- function(img, range = c(0.45, 0.55)
    ) {
    ids_negative = which(img < range[1])
    ids_positive = which(img > range[2])
    ids_hiperplane = which(img >= range[1] & img <= range[2])

    img[ids_negative] = 0
    img[ids_positive] = 1
    img[ids_hiperplane] = 0.5

    img
}

# This is a function to train and test the XOR problem
xor.test <- function(eta=0.1, threshold=1e-3) {

    # Loading the dataset "xor.dat"
    dataset = as.matrix(read.table("xor.dat"))

    # Building up the MLP architecture with random weights
    # and thetas. Observe we have two units at the input
    # layer (what is the number of input variables), we
    # have two units at the hidden layer (so we will have
    # two hyperplanes to shatter the space of examples as
    # expected), and we have a single unit at the output
    # layer to provide the answer as 0 or 1 (actually
    # values in range [0,1])
    model = mlp.architecture(input.layer.size = 2,
                             hidden.layer.size = 2,
                             output.layer.size = 1,
                             f.net = f)

    # Now we train the architecture "model" to build up
    # the "trained.model"
    trained.model = backpropagation(model, dataset,eta=eta,
                                    threshold=threshold)

    # Then we test the "trained.model" using the same
    # XOR dataset. For more complex problems, we will use
    # unseen examples.
    mlp.test(trained.model, dataset)

    # Building up hyperplanes to plot
    x = seq(-0.1,1.1,length=100)
    hiperplane_1 = outer(x,x,
```

```
            function(x,y) { cbind(x,y,1) %*%
                trained.model$layers$hidden[1,] } )

    hiperplane_2 = outer(x,x,
            function(x,y) { cbind(x,y,1) %*%
                trained.model$layers$hidden[2,] } )

    cat("Press enter to plot both hiperplanes...")
    readline()

    # Plotting the hyperplanes built at the hidden layer
    filled.contour(discretize.hiperplane(hiperplane_1) +
                   discretize.hiperplane(hiperplane_2))
}
```

We approach the problem XOR by calling function `xor.test()` with the default parameters. However, before invoking it, we need to load its source code using `source("mlp.r");` (using the R Statistical Software). After running `xor.test()`, the reader will observe the input examples and expected output classes, as shown next:

```
> xor.test()
Input data...
     [,1] [,2]
[1,]    0    0
[2,]    0    1
[3,]    1    0
[4,]    1    1
Expected output...
     [,1]
[1,]    0
[2,]    1
[3,]    1
[4,]    0
Enter to start running...
```

then, by typing "enter", messages about the training iterations are shown, until the average squared error converges to some value below the pre-defined threshold:

```
Squared error = 0.00100089
Squared error = 0.001000691
Squared error = 0.001000491
Squared error = 0.001000292
Squared error = 0.001000093
Squared error = 0.0009998944
Enter to start testing...
```

by typing "enter" again, the classification results are provided:

```
Input pattern = 0 0 Expected = 0 Predicted = 0.03463937
Input pattern = 0 1 Expected = 1 Predicted = 0.9699704
Input pattern = 1 0 Expected = 1 Predicted = 0.97008
Input pattern = 1 1 Expected = 0 Predicted = 0.03163266
Press enter to plot both hiperplanes...
```

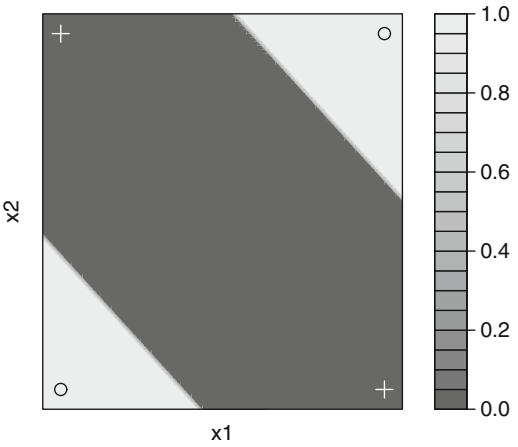


Fig. 1.50 The hyperplanes plotted after training the Multilayer Perceptron to solve the problem XOR

Observe the obtained outputs are not exactly the same as the expected ones, however they close enough. By setting a smaller threshold in Listing 1.9, one can obtain a better approximation to the expected classes. If the user types "enter" again, (s)he will see the two hyperplanes inferred during the training stage, as illustrated in Fig. 1.50.

For illustration purposes, consider the problem of Optical Character Recognition (OCR), in which the characters are represented using binary matrices:

$$\begin{matrix} 0&0&0&0&0&0&0\\ 0&0&1&1&1&0&0\\ 0&1&1&1&1&1&0\\ 0&1&1&0&1&1&0\\ 0&1&1&0&1&1&0\\ 0&1&1&1&1&1&0\\ 0&1&1&1&1&1&0\\ 0&1&1&0&1&1&0\\ 0&1&1&0&1&1&0\\ 0&0&0&0&0&0&0 \end{matrix} \quad \text{and} \quad \begin{matrix} 0&0&0&0&0&0&0\\ 0&1&1&1&1&0&0\\ 0&1&1&0&1&1&0\\ 0&1&1&0&1&1&0\\ 0&1&1&1&1&0&0\\ 0&1&1&0&1&0&0\\ 0&1&1&0&1&1&0\\ 0&1&1&0&1&1&0\\ 0&1&1&1&1&1&0\\ 0&0&0&0&0&0&0 \end{matrix},$$

in which the first represents character "A" and, the second, character "B". So, let MLP be trained to learn those characters. As first step, one needs to set the number of input units (or neurons) for this MLP instance. Observe matrices contain 10×7 cells, so there will be 70 neurons at the input layer. Since, no one knows[10] how to set the number of hyperplanes, i.e., number of units in the hidden layer, we could guess

[10]We can take advantage of the Statistical Learning Theory to set an adequate number of hyperplanes depending on the target problem. This is discussed in Chap. 2.

it by trying some number such as 3 or 5, what is fine for a first attempt. Finally, a single output neuron is used, so MLP could associate "A" to the output value 0 and "B" to 1. That is indeed possible, however what if we decide to "activate" one output neuron for every different example? In such situation, we should define two neurons at the output layer, then when "A" is given as input, MLP would produce the pair $(1, 0)$, and when "B" is received, it would output $(0, 1)$. In this manner, each output neuron represents one input character. By defining the number of output neurons as the number of classes, MLP tends to avoid as much as possible any class overlapping. Such a class mixing may happen due to the output layer combines the hyperplanes built in the hidden layer.

Listing 1.10 includes the previous source code (Listing 1.9) and adds up a new function to train and test the OCR problem. The reader must load this code, run function `ocr.test()` and attempt other parameters instead of the default ones.

Listing 1.10 Additional function to solve the OCR problem

```
source("mlp.r")

# Solving the Optical Character Recognition (OCR) problem.
ocr.test <- function(eta=0.1, threshold=1e-3) {

   # Loading the dataset
   dataset = as.matrix(read.table("ocr-asvector.dat"))

   # Loading a test set with unseen examples
   test.dataset = as.matrix(read.table("test-ocr-asvector.
       dat"))

   # Building up the architecture with 70 units at the input
   # layer, 5 units (so 5 hyperplanes) at the hidden layer
   # and 2 at the output layer.
   model = mlp.architecture(input.layer.size = 10*7,
              hidden.layer.size = 5,
              output.layer.size = 2, f.net = f)

   # Training
   trained.model = backpropagation(model,
             dataset,
             eta=eta,
             threshold=threshold)

   # Testing for unseen examples
   mlp.test(trained.model, test.dataset)
}
```

The files used in this problem are `ocr-asvector.dat` and `test-ocr-asvector.dat`. Both transform those previously presented binary matrices into binary vectors, and add the expected output values at the end. After calling function `ocr.test()`, we will see the following output:

```
Input data...
     [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
[1,]    0    0    0    0    0    0    0    0    0     1
[2,]    0    0    0    0    0    0    0    0    1     1

     [,11] [,12] [,13] [,14] [,15] [,16] [,17] [,18] [,19]
         [,20]
[1,]     1     1     0     0     0     1     1     1     1
         1
[2,]     1     1     0     0     0     1     1     0     1
         1

     [,21] [,22] [,23] [,24] [,25] [,26] [,27] [,28] [,29]
         [,30]
[1,]     0     0     1     1     0     1     1     0     0
         1
[2,]     0     0     1     1     0     1     1     0     0
         1

     [,31] [,32] [,33] [,34] [,35] [,36] [,37] [,38] [,39]
         [,40]
[1,]     1     0     1     1     0     0     1     1     1
         1
[2,]     1     1     1     0     0     0     1     1     0
         1

     [,41] [,42] [,43] [,44] [,45] [,46] [,47] [,48] [,49]
         [,50]
[1,]     1     0     0     1     1     1     1     1     0
         0
[2,]     0     0     0     1     1     0     1     1     0
         0

     [,51] [,52] [,53] [,54] [,55] [,56] [,57] [,58] [,59]
         [,60]
[1,]     1     1     0     1     1     0     0     1     1
         0
[2,]     1     1     0     1     1     0     0     1     1
         1

     [,61] [,62] [,63] [,64] [,65] [,66] [,67] [,68] [,69]
         [,70]
[1,]     1     1     0     0     0     0     0     0     0
         0
[2,]     1     1     0     0     0     0     0     0     0
         0

Expected output...
     [,1] [,2]
[1,]    1    0
[2,]    0    1
Enter to start running...
```

in which two vectors were provided, the first corresponding to the matrix representing character "A" and the second to "B". Next, the expected values are listed, activating either one of the output neurons according to the input character. After typing "enter", some output information similar to the following is produced:

```
Squared error = 0.001000569
Squared error = 0.001000473
Squared error = 0.001000378
Squared error = 0.001000283
Squared error = 0.001000188
Squared error = 0.001000092
Squared error = 0.0009999973
Enter to start testing...
```

which corresponds to the average squared error produced at every training iteration, until the stop criterion is reached, i.e., converging to threshold. Another "enter" will produce a result similar to:

```
Input pattern = 0 1 0 0 0 0 1 0 0 1 1 1 0 0 1 1 1 1 1 1 0 0
    0
    1 0 1 1 0 0 1 1 0 0 1 0 0 1 1 1 1 1 0 0 1 1 0 1 1 0 0 1
        1
    0 1 1 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0
Expected output = 1 0
Obtained output = 0.9632798 0.03591298

Input pattern = 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 1 1 1 1 1 0 1
    1
    0 0 1 1 0 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 0 1 1 0 1 1
        0
    0 1 1 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0
Expected output = 0 1
Obtained output = 0.05629617 0.9356224
```

which shows both input examples (in form of vectors), their expected and obtained outputs. The first input example should produce the following output pair $(1, 0)$ as it corresponds to a noisy version of character "A". In fact, it produced $(0.96327978, 0.03591298)$, being very close to the expected values once the first neuron is obviously activated. The second example should produce $(0, 1)$ and, in fact, it generated $(0.05629617, 0.93562236)$, respecting the idea of mostly activating the second neuron.

To improve visualization, those two input vectors are shown next, after being organized as binary matrices:

$$
\begin{matrix}
0\,1\,0\,0\,0\,0\,1 \\
0\,0\,1\,1\,1\,0\,0 \\
1\,1\,1\,1\,1\,1\,0 \\
0\,0\,1\,0\,1\,1\,0 \\
0\,1\,1\,0\,0\,1\,0 \\
0\,1\,1\,1\,1\,1\,0 \\
0\,1\,1\,0\,1\,1\,0 \\
0\,1\,1\,0\,1\,1\,0 \\
0\,1\,0\,0\,1\,1\,0 \\
0\,0\,1\,0\,0\,0\,0
\end{matrix}
\quad
\begin{matrix}
0\,0\,0\,0\,0\,0\,0 \\
0\,1\,1\,1\,1\,0\,0 \\
0\,1\,1\,1\,1\,1\,0 \\
1\,1\,0\,0\,1\,1\,0 \\
0\,1\,1\,1\,1\,0\,0 \\
0\,1\,1\,1\,1\,0\,0 \\
0\,1\,1\,0\,1\,1\,0 \\
1\,1\,0\,0\,1\,1\,0 \\
0\,1\,1\,1\,1\,1\,0 \\
0\,0\,0\,0\,0\,0\,0
\end{matrix}
$$

Both test examples contain some noise, however MLP is still capable of learning how to separate them out. We now suggest the reader to extend this problem to cover all English characters and digits, train and test the MLP for unseen examples (with some random noise, for example). We suggest as many neurons at the output layer as the number of possible input characters. About the number of hidden units, the reader may try different values in attempt to produce good results. More information will be provided on that matter throughout this book. Finally, the reader may download the datasets available at the UC Irvine Machine Learning Repository (http://archive.ics.uci.edu/ml), design, train and test different MLP instances on other classification tasks, such as Iris (http://archive.ics.uci.edu/ml/datasets/Iris) and Wine (http://archive.ics.uci.edu/ml/datasets/Wine). The reader may also perform experiments using the MNIST handwritten digit database and compare the MLP results against others reported in literature [7].

1.6 Concluding Remarks

This chapter presented a brief review on Machine Learning (ML), mainly focusing on supervised learning algorithms. This type of learning was tackled due to it relies on the theoretical foundation provided by the Statistical Learning Theory (SLT). After introducing the main aspects of the SLT, such as its assumptions, the concept of loss function, the empirical and expected risks, more information was provided about how the Bias-Variance Dilemma is considered in the context of ML and, finally, introduced two well-known supervised learning algorithms (Perceptron and Multilayer Perceptron) for illustration purposes. All those concepts are essential to support the next chapters.

1.7 List of Exercises

1. Run the Perceptron on the problem `XOR` and assess the obtained results. Take your own conclusions why this algorithm cannot learn in such scenario;
2. Randomly select 30% of the examples contained in the Iris dataset to compose your test set. Then use the remaining 70% to form your training set. Start training an Multilayer Perceptron using three hidden neurons and, then, compute the error for the same training examples and, afterwards, the error taking the test examples. Observe how those two errors deviate one another as the number of hidden neurons is increased. A good enough model has to provide close enough errors. When their absolute deviation grows, it means MLP is considering too many (or too few if reducing) hidden units.[11]
3. Using the Wine dataset, train and test the Multilayer Perceptron. Then, assess the obtained results.[12]
4. Based on the previous exercise, notice how the quasi-convexity present in the MLP squared-error function affects training and testing. So, normalize all Wine input attributes in range [0, 1] and observe the new results.
5. Consider the MNIST database to train and test a Multilayer Perceptron algorithm with a varying number of hidden neurons.[13] MNIST is already segmented into training and test sets. Compute errors on the test set and compare to the ones listed in http://yann.lecun.com/exdb/mnist.

References

1. C.M. Bishop, *Pattern Recognition and Machine Learning*. Information Science and Statistics (Springer, New York, 2006)
2. S. Boyd. L. Vandenberghe, *Convex Optimization* (Cambridge University Press, New York, 2004)
3. G. Carlsson, F. Mémoli, Characterization, stability and convergence of hierarchical clustering methods. J. Mach. Learn. Res. **11**, 1425–1470 (2010)
4. M.E. Celebi, K. Aydin, *Unsupervised Learning Algorithms* (Springer International Publishing, Berlin, 2016)
5. C.M. Grinstead, L.J. Snell, *Grinstead and Snell's Introduction to Probability* (American Mathematical Society, Providence, 2006), Version dated 4 July 2006 edition
6. S. Haykin, *Neural Networks: A Comprehensive Foundation*, 3rd edn. (Prentice-Hall, Upper Saddle River, 2007)
7. Y. LeCun, C. Cortes, MNIST handwritten digit database (2010). http://yann.lecun.com/exdb/mnist/

[11] Download the Iris dataset from the UCI Machine Learning Repository available at archive.ics.uci.edu/ml.

[12] Download the Wine dataset from the UCI Machine Learning Repository available at archive.ics.uci.edu/ml.

[13] The MNIST database is available at http://yann.lecun.com/exdb/mnist.

8. T.M. Mitchell, *Machine Learning*, 1st edn. (McGraw-Hill, New York, 1997)
9. H.-L. Nguyen, Y.-K. Woon, W.-K. Ng, A survey on data stream clustering and classification. Knowl. Inf. Syst. **45**(3), 535–569 (2015)
10. R Development Core Team, *R: A Language and Environment for Statistical Computing* (R Foundation for Statistical Computing, Vienna, 2008). ISBN 3-900051-07-0
11. F. Rosenblatt, The perceptron: a perceiving and recognizing automaton, Technical report 85-460-1, Cornell Aeronautical Laboratory, 1957
12. W.C. Schefler, *Statistics: Concepts and Applications* (Benjamin/Cummings Publishing Company, San Francisco, 1988)
13. B. Scholkopf, A.J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond* (MIT Press, Cambridge, 2001)
14. R.M.M. Vallim, R.F. de Mello, Unsupervised change detection in data streams: an application in music analysis. Prog. Artif. Intell. **4**(1), 1–10 (2015)
15. V.N. Vapnik, *Statistical Learning Theory*. Adaptive and Learning Systems for Signal Processing, Communications, and Control (Wiley, Hoboken, 1998)
16. V. Vapnik, *The Nature of Statistical Learning Theory*. Information Science and Statistics (Springer, New York, 1999)
17. U. von Luxburg, B. Schölkopf, *Statistical Learning Theory: Models, Concepts, and Results*, vol. 10 (Elsevier North Holland, Amsterdam, 2011), pp. 651–706
18. T.J. Ypma, Historical development of the Newton-Raphson method. SIAM Rev. **37**(4), 531–551 (1995)